

Data-Driven Safety & Security of Cyber-Physical Systems

Omanshu Thapliyal

Flight Dynamics & Control/Hybrid Systems Lab

School of Aeronautics and Astronautics

Purdue University

Co-advisor: Dr. Inseok Hwang, Co-Advisor: Dr. Arthur Frazho

Committee: Dr. Dengfeng Sun, Dr. Shaoshuai Mou

Outline

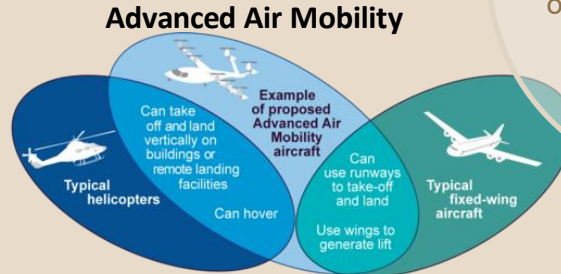
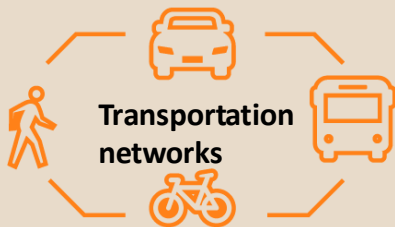
- Background and Motivation
- Research Gaps
- Research Objectives
- Post-Prelim Results
- Conclusion

Background and Motivation

Cyberphysical Systems

CPSs integrate computation, control, and communication into physical objects and infrastructure, connecting them to the Internet and to each other.

Examples include:



Source: GAO. | GAO-23-105188

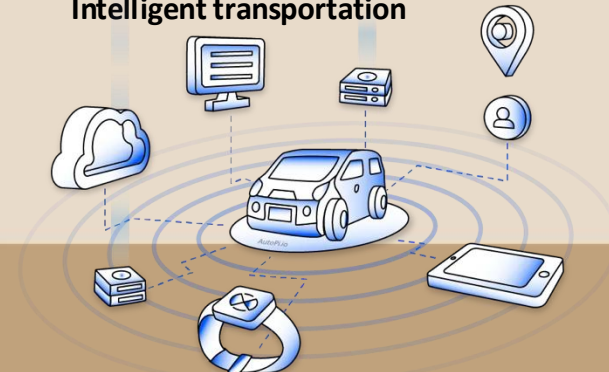
Smart grid



Internet of Things



Intelligent transportation



RL, IRL/IOC, data-driven control, etc.

Optimization, planning, Routing, resource management, Operations research
Computation

CPSs

IoT, UAM, Power Grid, NAS, etc.

Classical control, system ID, stability, controllability, optimal control
Control

Protocols, filtering, estimation, cryptography

Communication

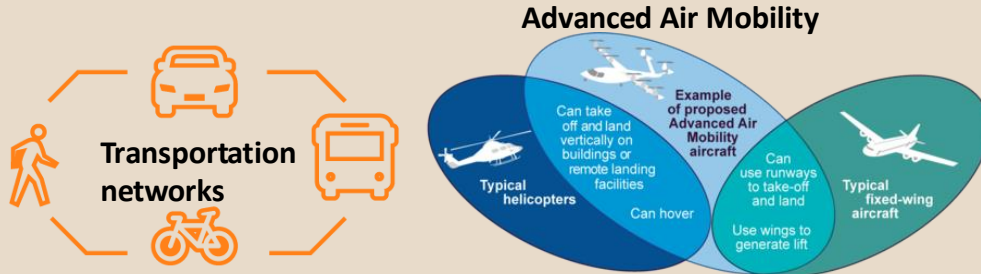
Distributed optimization, Distributed learning, MARL, etc.

Distributed, Wireless control, Cybersecurity

Complexity in CPS Operating Environments

CPSs are characterized by complexity in operating environments due to:

- Complex Control Systems
- Data Availability and Reliance of Data-in-the-loop Methods
- Safety-Critical Operating Environments



Source: GAO. | GAO-23-105188

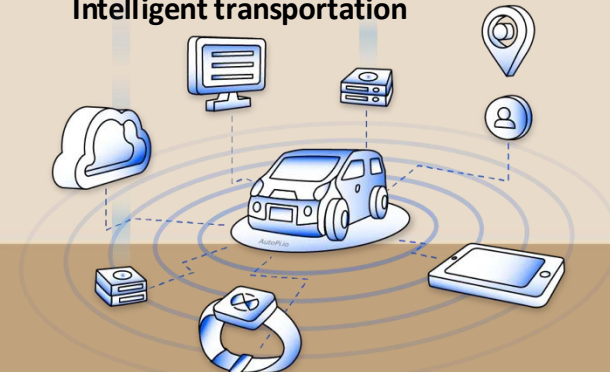
Smart grid



Internet of Things



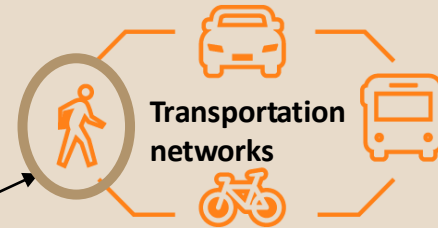
Intelligent transportation



Complexity in CPS Control Problems

CPS control problems get complexity from:

- Parameter Variation/Unknown models
 - E.g., decision models for passengers in multi-modal transportation networks
- Multi-agent interactions
 - E.g., smart grid distributed control problems
- Unknown/unseen operational scenarios
 - E.g., AAM operating environments with unknown, noncooperative UAVs
- Unstructured Control Problems
 - E.g., simultaneous localization & mapping for autonomous driving



Smart grid

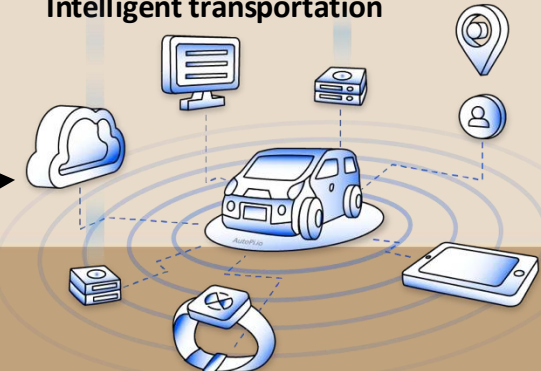


Advanced Air Mobility



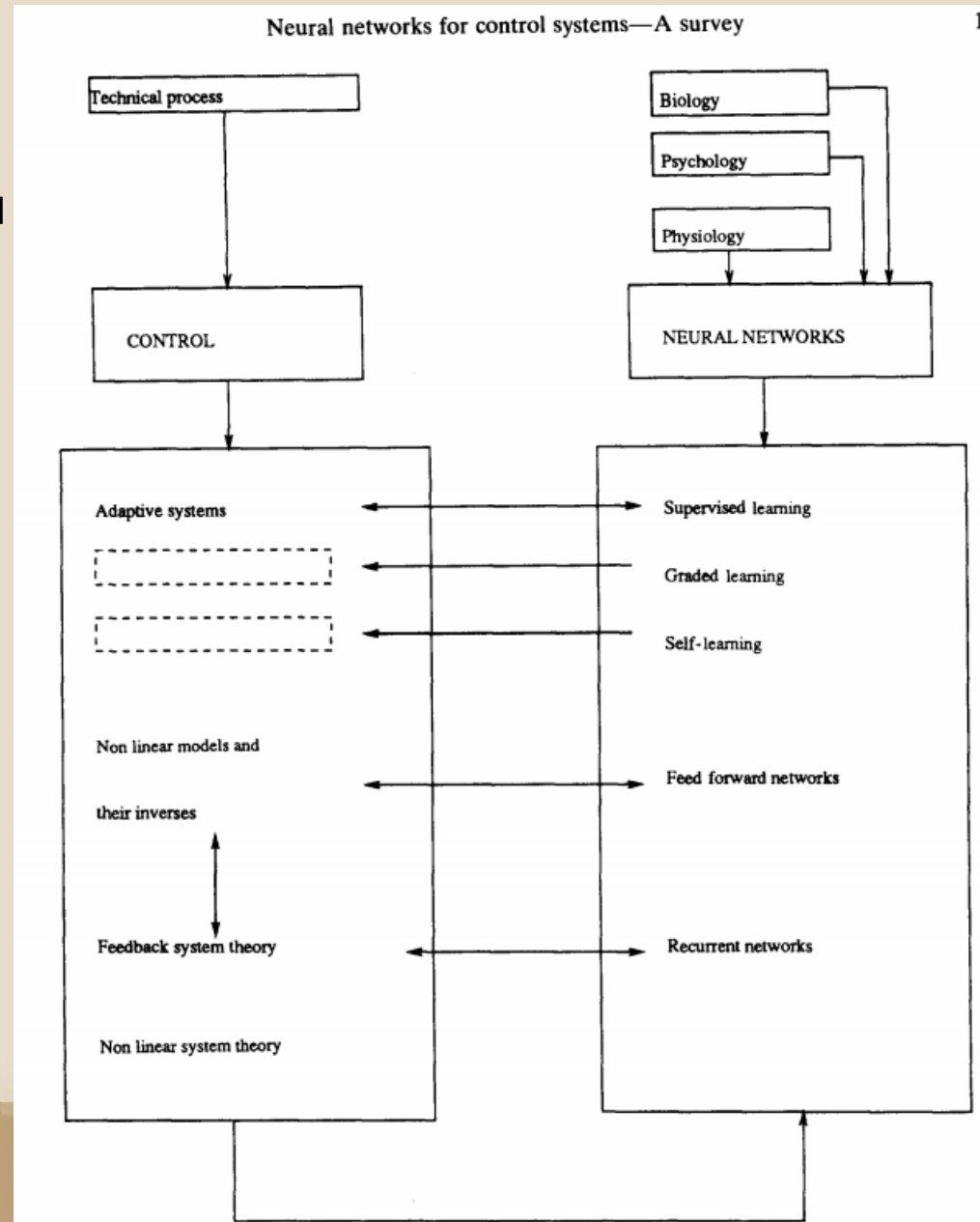
Source: GAO. | GAO-23-105188

Intelligent transportation



Data Availability for Complex Systems

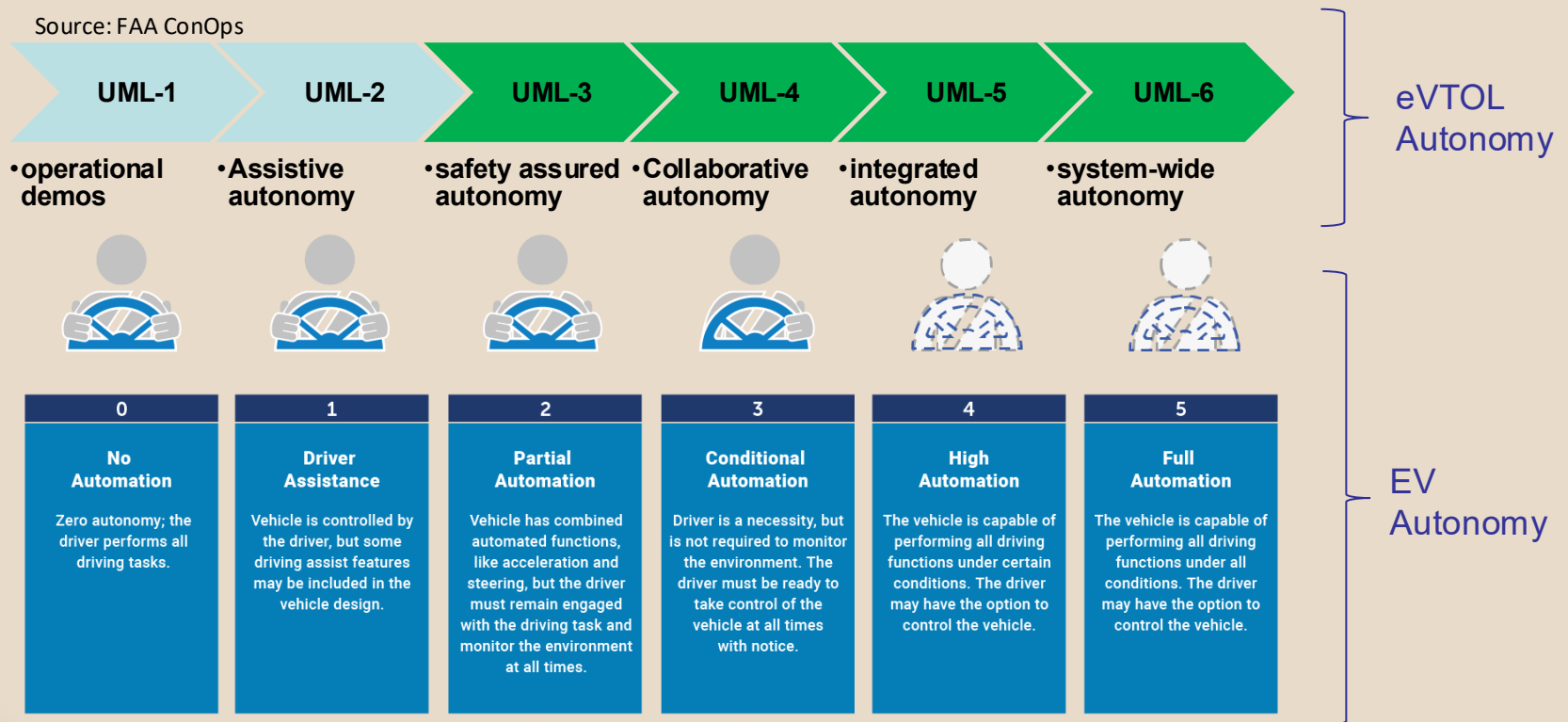
- Reliance on communication
- Data, in general, is cheaper to obtain and in abundance
- Data-driven methods for controls overtaking model-based control strategies
- General relationship between the fields of control science and neural networks, according to Hunt et al, 1992 (right)
 - Over time, some 'blank boxes' have been populated, e.g., self learning $\leftrightarrow$ fuzzy-reinforcement learning-based control [2]



Safety-Critical Operational Requirements

- Despite the prevalence of data-driven methods and complex dynamical interactions, such systems are often required to operate in safety-critical scenarios
- E.g., increasing levels of autonomy in UAM and autonomous driving technologies

Source: FAA ConOps



Source: Society of Automotive Engineers Automation Levels

- As a result of increasing levels of autonomy granted to the CPS control, **safety is one of the biggest cross-cutting barriers** in emerging control problems, **especially in mobility and air transportation**

Future Mobility Challenges

Psychological barriers
Psychological barriers related to flying in a fully autonomous aircraft

Safety
Amassing a flawless operational and mechanical safety record

Regulations
Formulating the regulations for pilotless vehicles, airworthiness certifications, and the need for a pilot's license

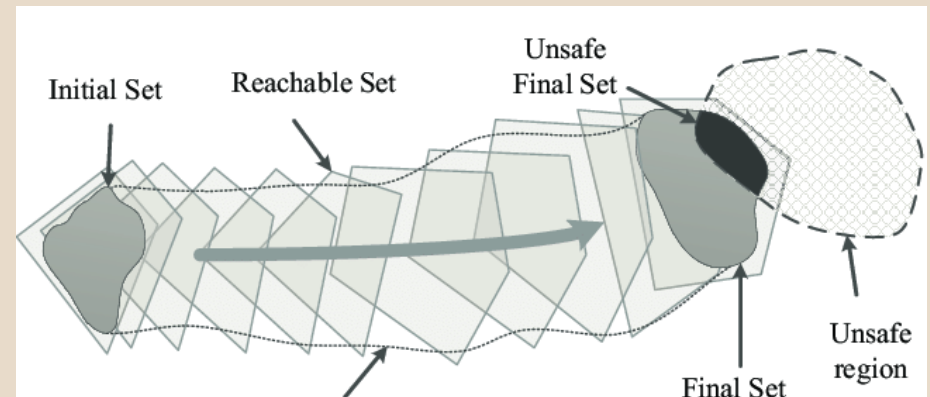
Technology maturity
Implementing efficient energy management systems, onboard sensors, collision detections systems, and advanced technologies such as artificial intelligence

Infrastructure
Constructing takeoff and landing zones, parking lots, charging stations, and vertiports

Air traffic management
Creating a robust air traffic management system integrated with other modes of transport

Source: Deloitte Analytics

- Set properties for such applications (e.g., reachability and viability) are usually used as a metric for characterizing safe/unsafe operational regions of the state space
- Real-time applications of data-driven control methods requires real-time computation of safety properties



Reachable set for a set of trajectories under certain control inputs [4]

Research Gaps

Existing Reachable Set Computation Methods

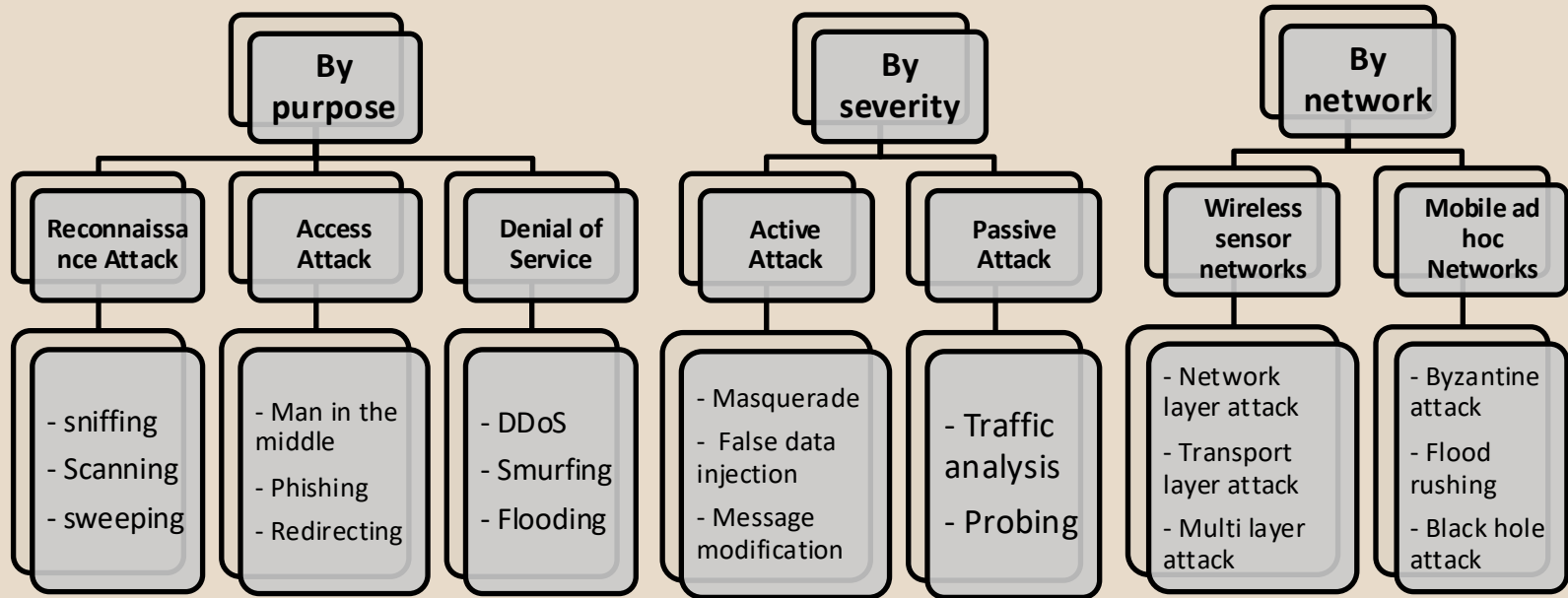
	Hamilton Jacobi PDE	Polytopic Approx.	Polytopic (Numerical Integration)	Zonotope Propagation	Verification methods for NNs
Computation cost	High	Low	Low	Low	High
Real-time applicability	Low	High	High	High	Low
Extendable to data- based plant models	No	No	No	No	Yes
Handle inputs	Yes	Yes	Yes	No	Yes
Requires exact/high fidelity model	Yes	Yes	Yes	Yes	Yes
Handle Nonlinearities	Yes	No	Yes	No	Yes

we want a reachable set computation method with...

real-time applicability low computational complexity Usable with data-based methods/NNs Works with unknown System model

...given system's trajectory data

Data-Driven Security Methods: *Existing Cyberattacks*



- Complex, networked control systems are subject to combined attacks of different types, e.g. probing + Byzantine + False data Injection
- More 'patient' attackers can exploit further vulnerabilities by probing for more data
- Side channel attackers can help attackers collect more data on the system's internal state

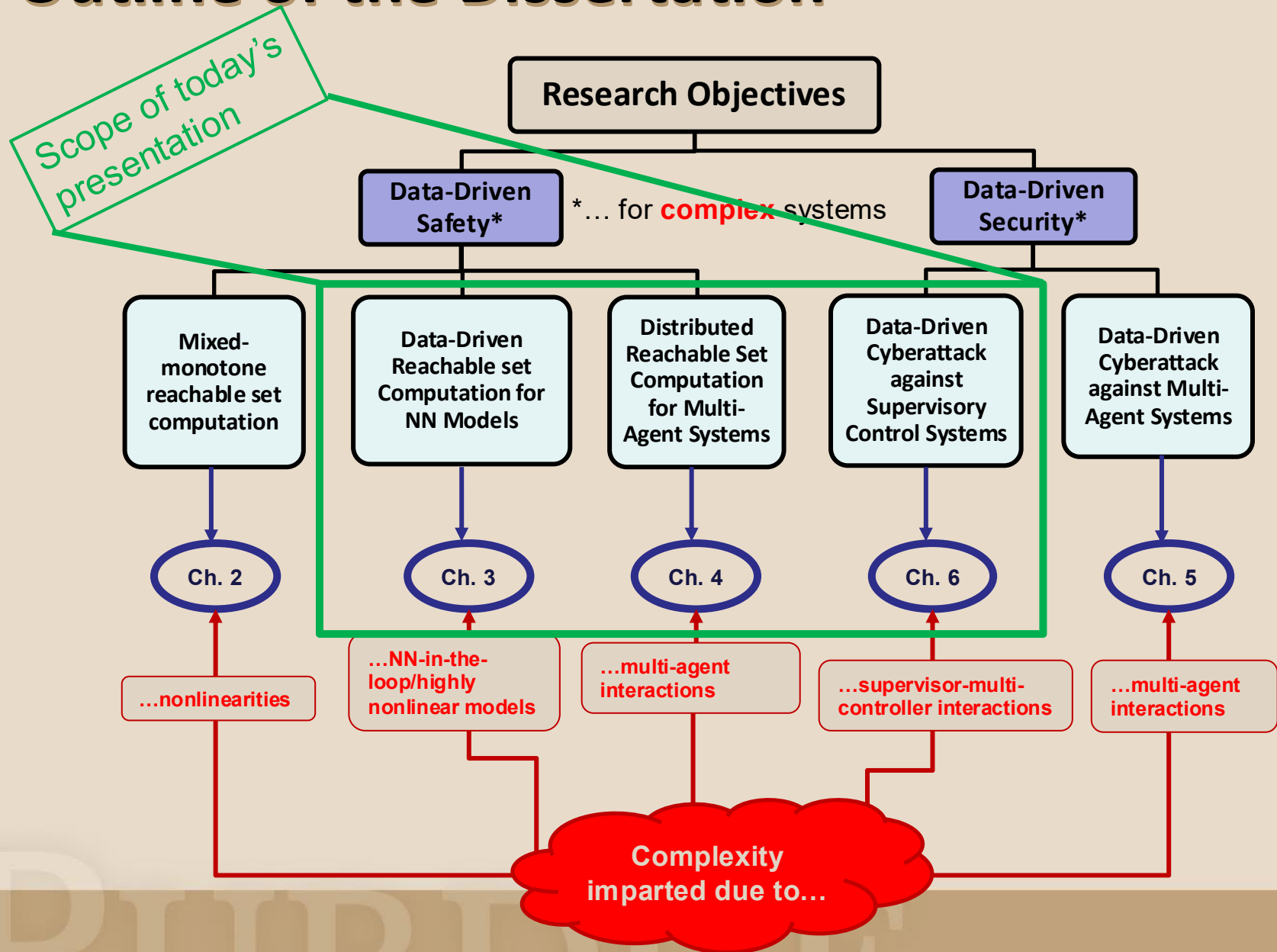
How can the attacker utilize collected data for worse/combined attacks?

Conversely, can the defender utilize the data being probed for?

Primary Objective: to study data-driven approaches to safety, and data-driven vulnerabilities against cyberattacks, of complex control systems

- **Safety:** *Given enough data, can we comment upon system safety?*
 - Model-based approaches can no longer be used → use data-driven models
 - Can we approximate reachable sets for 'discovered models' to determine safety?
- **Security:** *Given enough data, in what ways can an attacker violate system security?*
 - If a '*patient attacker*' can probe/side channel for system data, in what ways can system security be compromised?
 - What are some possible defense mechanisms against '*patient cyberattacks*'?

Outline of the Dissertation



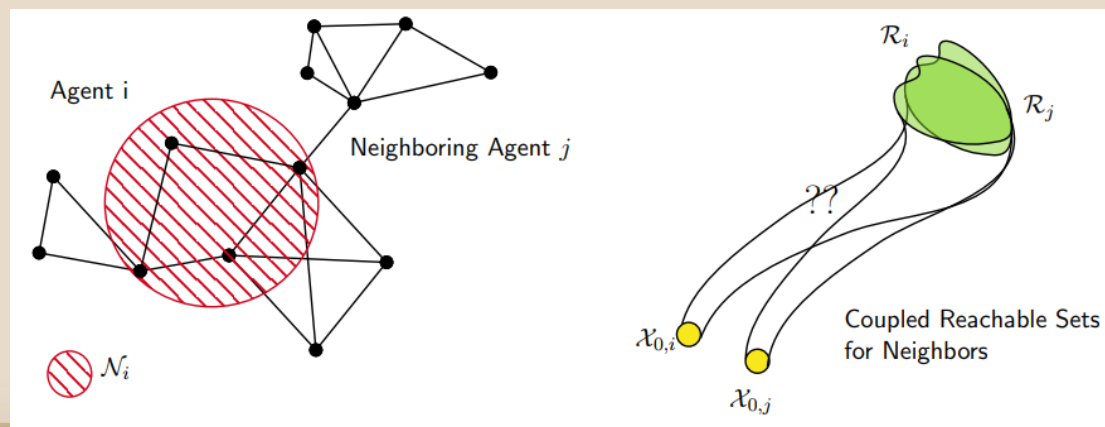
Outline

- Background and Motivation
- Research Gaps
- Research Objectives
- Post-Prelim Results
 - **Distributed Reachable Set Computation for Multi-Agent Systems**
 - Data-Driven Reachable set Computation for Neural Network Models
 - Data-Driven Cyberattack against Multi-Agent Systems
- Conclusion

Distributed Reachable Set Computation for Multi-Agent Systems

Motivation

- For dynamical systems with dedicated computational resources, a centralized reachability problem is limited only by the accuracy of the dynamical model
- Multi-Agent systems (MAS) are cheaper components in a bigger network of agents, internet-of-things network, or a system-of systems
- MASs have limited computational capabilities
 - This is the most severe bottleneck in computing properties of an MAS in a distributed manner
- The sensed and communicated information from the neighborhood of an agent affects its own reachable sets in non-trivial ways



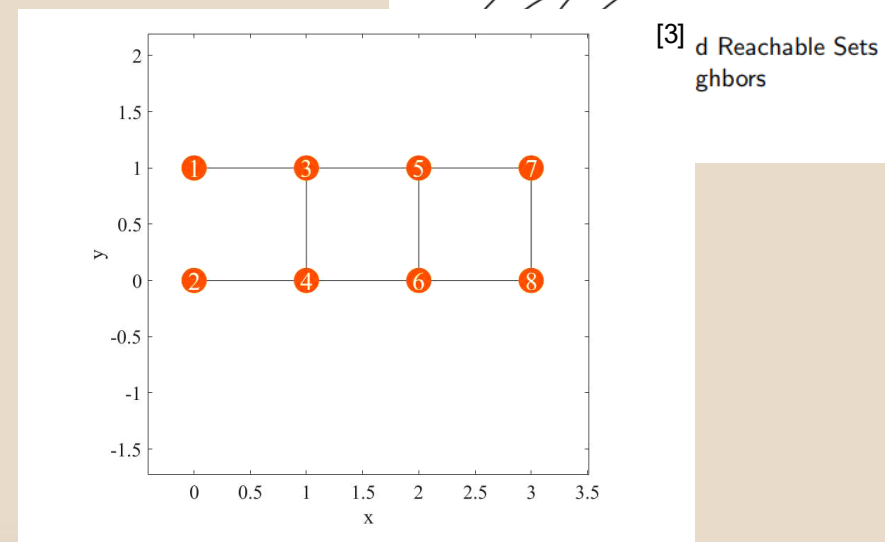
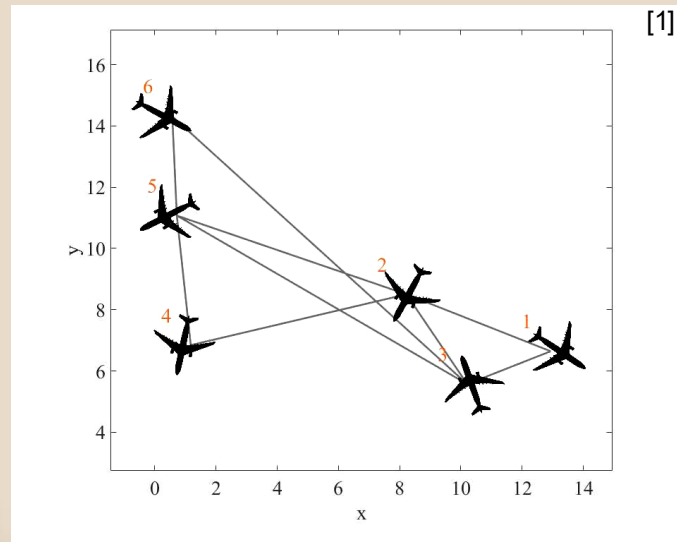
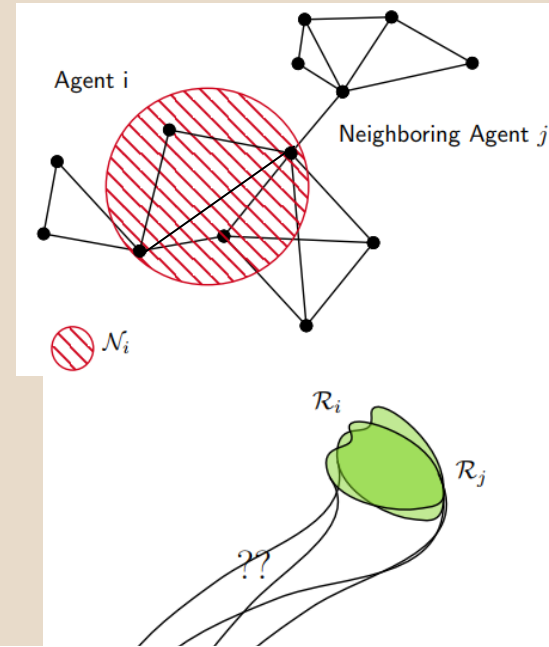
Consider a network of agents whose states are coupled by a state feedback control law

$$\dot{x}_i(t) = A_i x_i(t) + B_i u_i(t) + B_{1,i} w_i(t)$$

where $u_i(t) = K_i x_i(t) + \sum_{j \in \mathcal{N}_i} K_j x_j(t)$

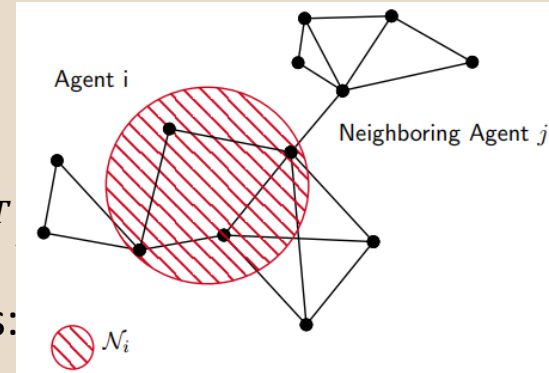
where the agents are connected according to some graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, with a graph adjacency $\mathcal{A}$, an initial state set $\mathcal{X}_{0,i}$ for each agent i , and unknown, exogenous control set $w_i \in \mathcal{W}_i$

- ...but why such a control structure?
- Because it models many realistic NCS problems:



- Distributed reachability problem is to compute $\mathcal{R}_i(\tau; \mathcal{X}_{0,i}, \mathcal{W}_i)$ where the i^{th} agent knows only $(A_i, B_i, B_{1,i}, \mathcal{N}_i)$
- We can define the stacked state LTI system using $\xi(t) = [x_1(t)^T \dots x_N(t)^T]^T$ and $\mathbb{B}(t) = \text{diag}\{B_{1,i}\}$, $W(t) = [w_1(t)^T, \dots, w_N(t)^T]^T$ as:

$$\dot{\xi}(t) = A(t)\xi(t) + \mathbb{B}(t)W(t)$$
- We can form an equivalent problem to compute:



$$\mathcal{R}^\xi(\tau; \Xi_0) := \left\{ \xi(\tau), \text{s.t. } \dot{\xi}(t) = A\xi(t) + \mathbb{B}W(t), \xi_i(0) \in \Xi_0, W(t) \in \mathcal{W} \right\}$$

$$\text{where } \Xi_0 := \prod_{i=1}^N \mathcal{X}_{i,0}, \mathcal{W} := \prod_{i=1}^N \mathcal{W}_i$$

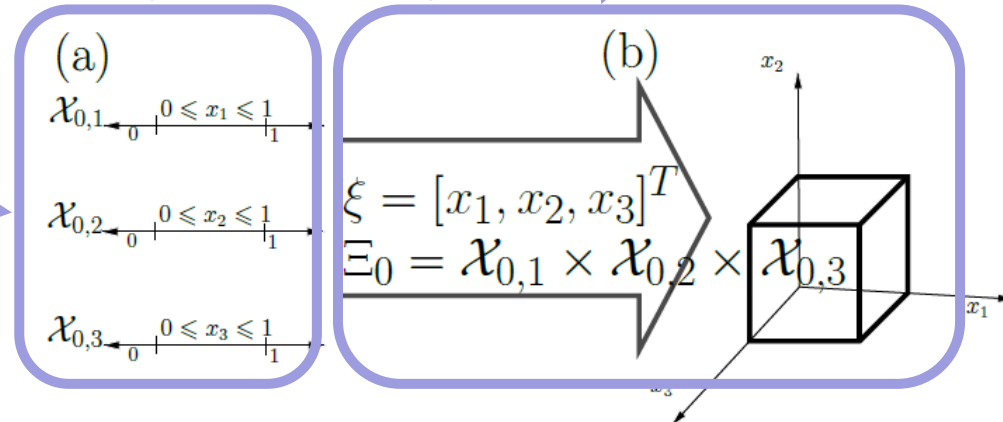
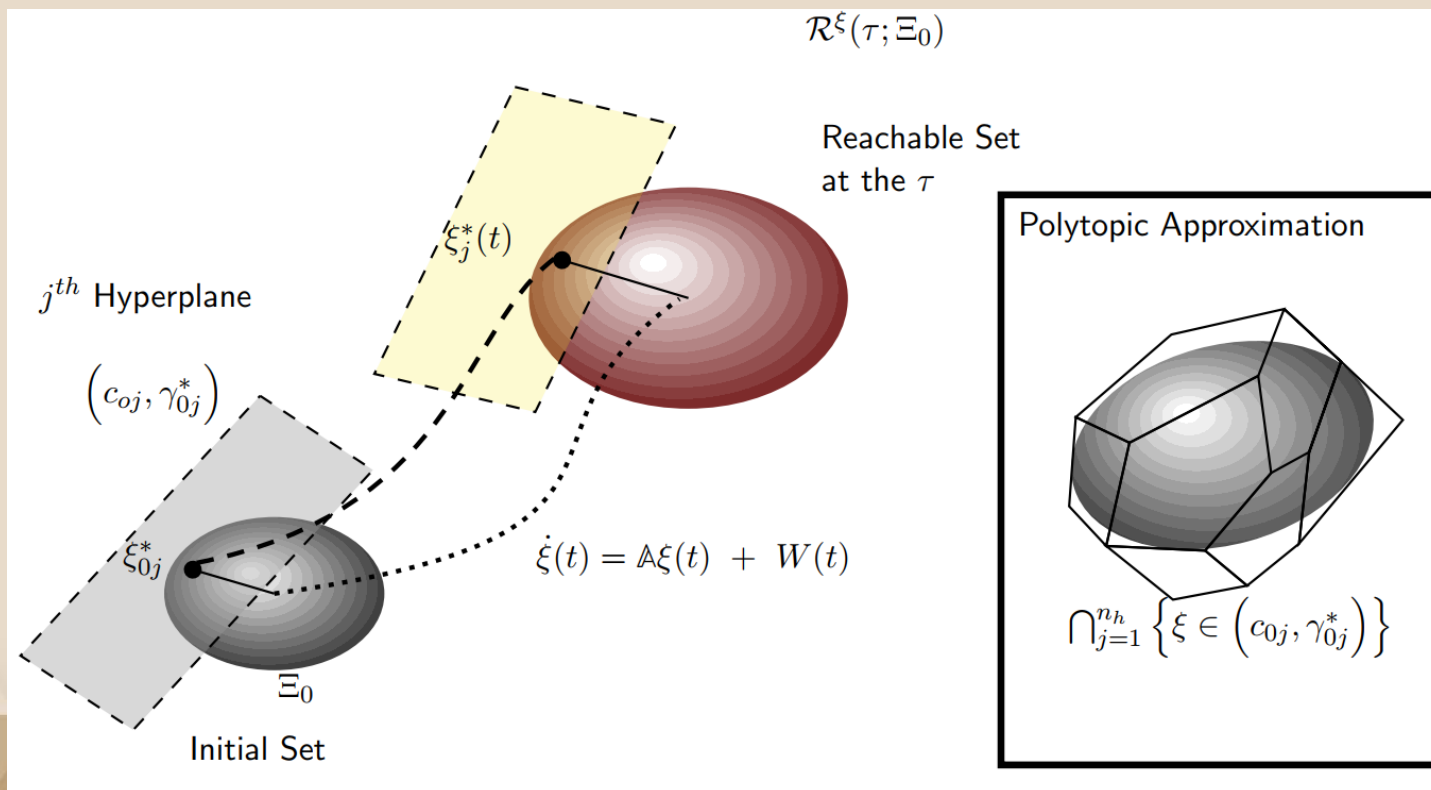


Figure 4.3. (a) Distributed, and (b) centralized structures of the polytope

- WLOG, we assume $\mathcal{W}$ is a polytope, and Ξ_0 is finite, thus can be bounded by a polytope (let it be an n_h -faced polytope)
- Let (c_{0j}, γ_{0j}^*) be the j^{th} hyperplane supporting Ξ_0 at contact point ξ_{0j}^*

$$\langle c_{0j}, \xi_{0j} \rangle \leq \gamma_{0j}^* = \langle c_{0j}, \xi_{0j}^* \rangle, \forall \xi_{0j} \in \Xi_0$$
- Consider the evolutions of only the supporting points $\xi_i^*(t)$ s.t.**

$$\mathcal{R}^\xi(\tau) = \bigcap_{j=1}^{n_h} \{ \xi(t) \in (c_j(t), \gamma_j^*(t)) \}$$



The propagation of supporting hyperplane contact points gives reachable set polytopes by solving an optimal control problem [10,11]

$$W_j^*(\tau) = \arg \max \{ \langle \lambda_j^*(\tau), \mathbb{A} \xi_j^*(\tau) + W_j \rangle, W_j \in \mathcal{W} \}$$

for costate vector $\dot{\lambda}_j^* = -\mathbb{A}^T \lambda_j^*$ s.t. $\lambda_j^*(0) = c_{0j}$ and associated state transition $\Phi_{-\mathbb{A}^T}$

Polytopic reachability is then comprised of steps [due to Theorem 4.3.1]:

(S1): Note $\lambda_j^*(0) = c_{0j}$, then propagate the costate as $\lambda_j^*(\tau) = \Phi_{-\mathbb{A}^T}(\tau; 0) \lambda_j^*(0)$.

(S2): Compute $W^*(\tau) = \arg \max \{ \langle \lambda_j^*(\tau), W_j \rangle, W_j \in \mathcal{W} \}$ which can be computed using step 1 as $\arg \max \{ \langle \Phi_{-\mathbb{A}^T}(\tau; 0) \lambda_j^*(0), W_j \rangle, W_j \in \mathcal{W} \}$.

(S3): Propagate the contact point as:

$$\xi_j^*(\tau) = \Phi_{\mathbb{A}}(\tau; 0) \xi_{0j}^* + \int_0^\tau \Phi_{\mathbb{A}}(\tau; s) W(s) ds \quad (37)$$

(S1)-(S3) must be carried out in a distributed way s.t. agent i knows only $(A_i, B_i, B_{1,i}, \mathcal{N}_i)$

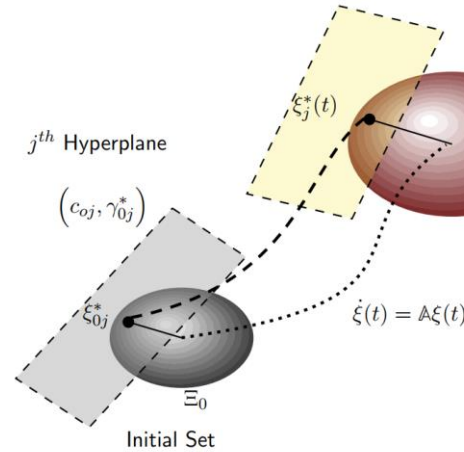
19 **Theorem 4.3.1** (Polytopic Reachability [35]). *Let hyperplane $H_j = (c_j(t_0), \gamma_j(t_0))$ support the initial set Ξ_0 at time t_0 , at a point $\xi_j^*(t_0)$. Then the point of contact $\xi_j^*(t_0)$ evolves as:*

$$\dot{\xi}_j(\tau) = \mathbb{A}\xi_j(\tau) + \mathbb{B}W_j^*(\tau) \quad (4.10)$$

where $W_j^*(\tau)$ solves the optimal control problem:

$$W_j^*(\tau) = \arg \max_{W \in \mathcal{W}} \{ \langle \lambda_j^*(\tau), \mathbb{A}\xi_j^*(\tau) + \mathbb{B}W \rangle \} \quad (4.11)$$

$$\dot{\lambda}_j^*(\tau) = -\mathbb{A}^T \lambda_j^*(\tau) \text{ s.t. } , \lambda_j^*(t_0) = c_j(t_0), t_0 \leq \tau \leq t \quad (4.12)$$



That is, for costate $\lambda_{j,i}$ associated with the i^{th} agent, for the j^{th} hyperplane,

- Consider Laplace transform of the costate differential equation
- For time invariant, undirected graph $\mathbb{A}(t) = \mathbb{A} = \mathbb{A}^T$
- $\rightarrow$ linear equation where each agent is aware of row partitions of $\mathbb{A}$, say, $\mathbb{A}_i$
- **Use distributed linear equation solver for (S1) and (S3)^[13]**
- [Distributed Linear Equation] $\lambda_{j,i}(\tau)$ can be solved for using linear updates on $\hat{\lambda}_{j,k}(\tau)$ s.t. $k \in \mathcal{N}_i$,

where the t^{th} update is $\hat{\lambda}_{j,i} \leftarrow \sum_{k \in \mathcal{N}_i} \alpha_k \hat{\lambda}_{j,k}(\tau)$ as $t \rightarrow \infty$ ^[13]

- α_k for each agent are computed using repeated projection of local solution $\hat{x}_i$ onto the kernel of $\mathbb{A}_i$

- $\therefore$ propagating costate $\rightarrow$ solving $Ax = b$ distributed across $\mathcal{G}$ as follows:

(a)

$$\mathbb{A} = \begin{bmatrix} \ddots & & \\ & A_i + B_i K_i + B_i \sum_{j \in \mathcal{N}_i} K_{ij} & \\ & & \ddots \end{bmatrix}, \quad \mathbb{B} = \begin{bmatrix} B_{1,1} & & \\ & \ddots & \\ & & B_{1,i} & \\ & & & \ddots \end{bmatrix}$$

(b) $Ax = b$

For solving optimal control in (S2)...

- Let $(\beta_1, \dots, \beta_k)$ be the vertices of the polytope bounding $\mathcal{W}$, due to Pontryagin, (S3) optimal control problem is a discrete problem $W_j^*(\tau)$ solves maximum of

$$\langle \Phi_{-\mathbb{A}^T}(\tau; 0) \lambda_j^*(0), W_j \rangle \text{ for } W_j \in \{\beta_1, \beta_2, \dots, \beta_k\}$$

- For strongly connected $\mathcal{G}$, in each round of communication, (S3) can be solved by simply cycling through the vertex sets what bound $\mathcal{W}$

$\mathcal{R}_i(\tau; \mathcal{X}_{0,i}, \mathcal{W}_i)$ can be computed in a distributed way, without knowing
 neighboring agents' dynamics for time-invariant, undirected $\mathcal{G}$

Extending Distributed Reachable Set Computation: time varying graphs $\mathbb{A} \rightarrow \mathbb{A}(t)$

Let $\mathcal{G}$ be the time-varying, undirected graph, given by $\mathcal{G}(t) = \{\mathcal{E}(t), \mathcal{V}(t)\}$

- $\rightarrow$ the graph adjacency varies as $\mathcal{A}(t) \Rightarrow$ we get a linear time-varying system $\mathbb{A}(t), \mathbb{B}$
- Since $\mathcal{G}(t)$ is undirected, $\mathcal{A} = \mathcal{A}^T$ at all times $\Rightarrow \mathbb{A}(t) = \mathbb{A}(t)^T$
- To propagate the costate and state, we require to calculate $\Phi^{\mathbb{A}}(\tau; 0)$
(instead of computing costate propagation as $\Phi^{\mathbb{A}^T}$)

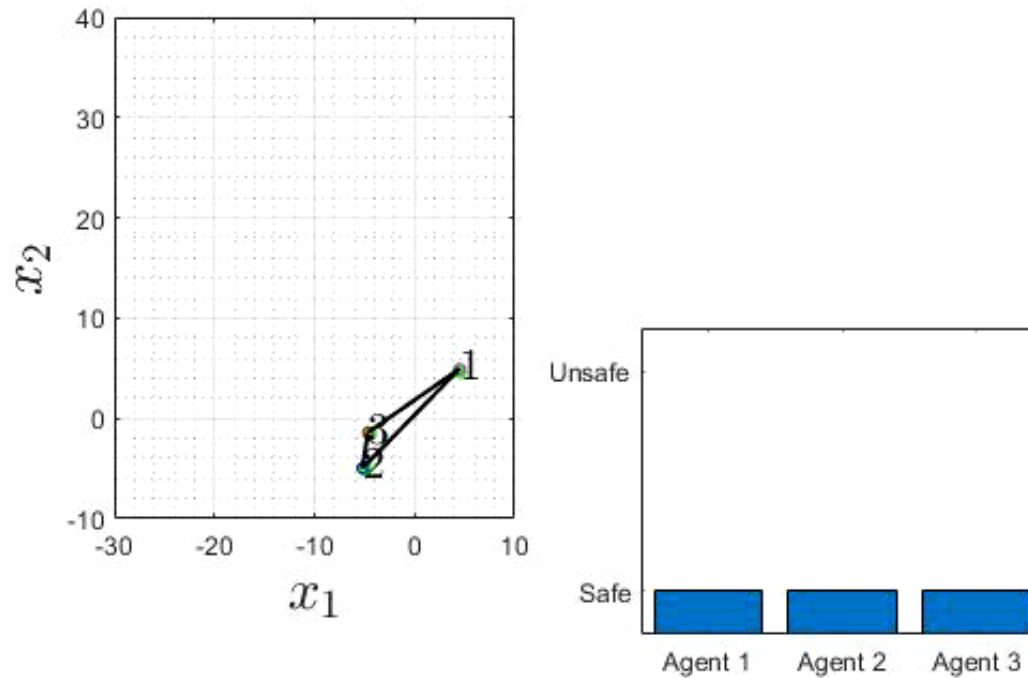
Lemma 3. *If the graph $\mathcal{G}(t)$ is (repeatedly jointly) strongly connected, the iterations in (4.20) converges to the solution to the equation $Ax = b$ exponentially fast.*

- $\therefore$ **Above can be carried out in a distributed manner for $\mathcal{G}(t)$ repeatedly jointly strongly connected at all times** (due to Theorem 4.3.2, which follows from Theorem 1 due to Mou et. al. in [13])

Graph composition is strongly connected over time

Example:

- 3 agent network + double integrator dynamics
- Reachable sets computed in a distributed manner are utilized to predict a safety over a horizon of $0.5s$.



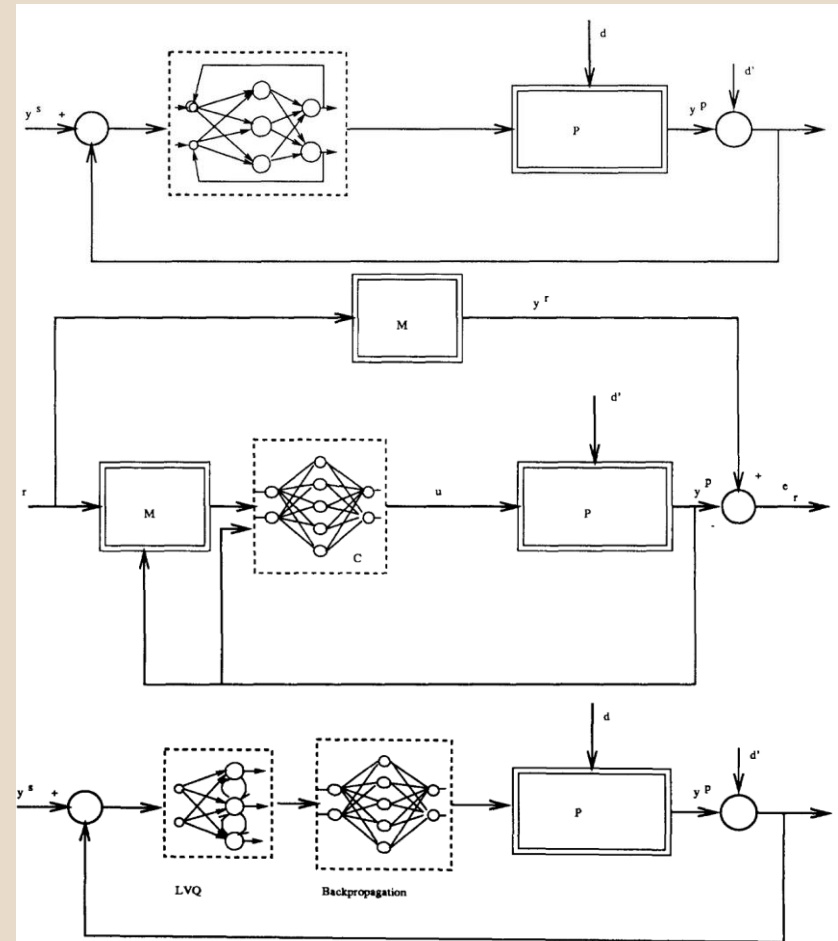
Outline

- Background and Motivation
- Research Gaps
- Research Objectives
- Post-Prelim Results
 - Distributed Reachable Set Computation for Multi-Agent Systems
 - **Data-Driven Reachable set Computation for Neural Network Models**
 - Data-Driven Cyberattack against Multi-Agent Systems
- Conclusion

Data-Driven Reachable set Computation for Neural Network Models

Motivation

- Neural Networks (NNs) are applied in the entire controls process, e.g.
 - system identification,
 - output tracking,
 - control synthesis,
 - state estimation, and
 - devising supervisory control logic
- Despite this, computing reachable sets for learned models, either requires intricate knowledge of connectivity of individual units in a NN, or require solving expensive mixed-integer problems
- Model-based methods can not be used directly on learned models due to NN models having obscured nonlinearities

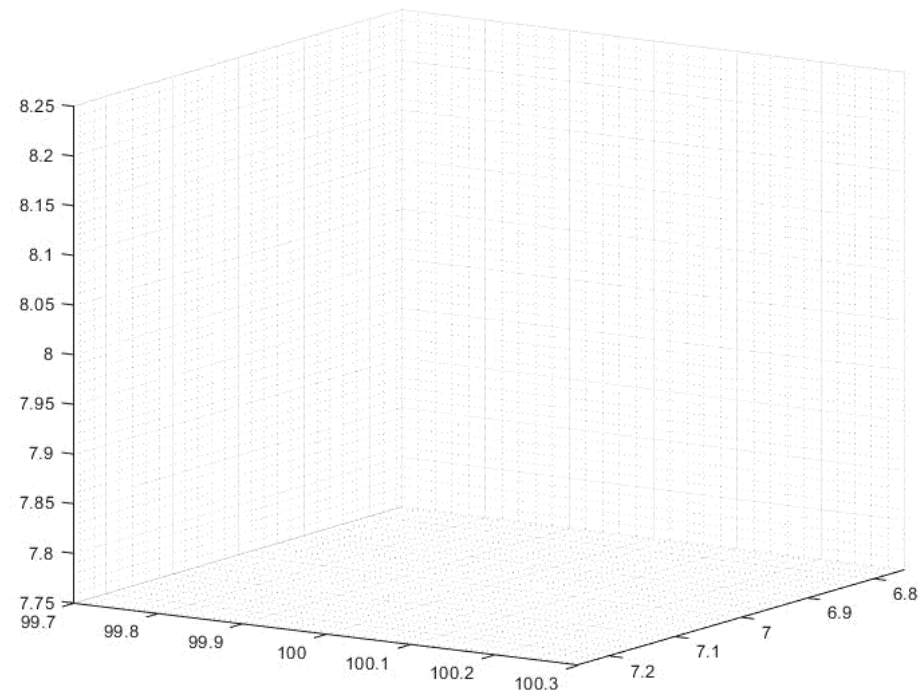
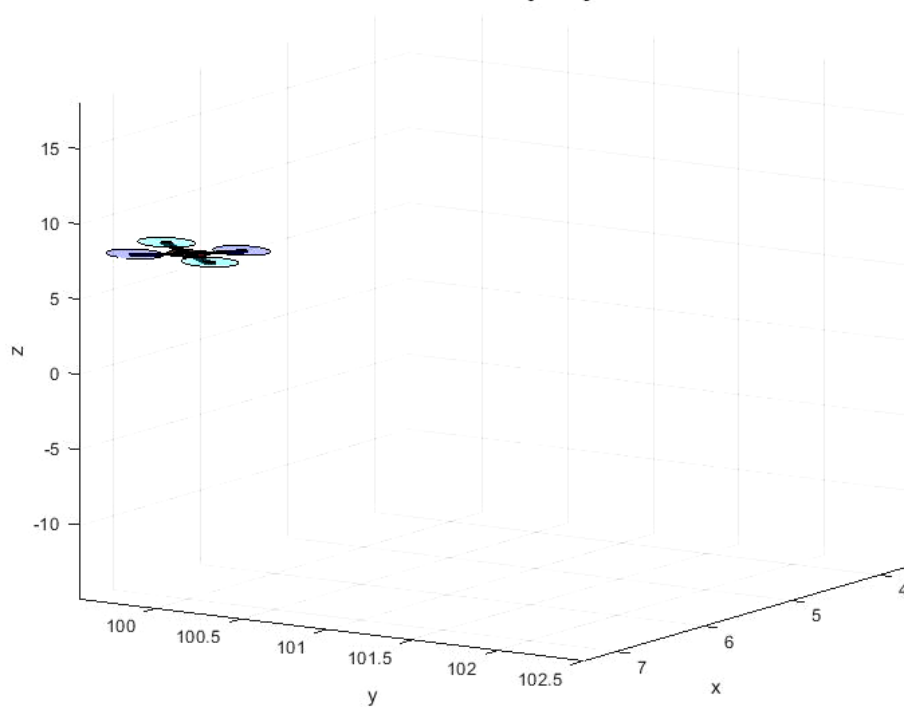


NNs used as (top) a controller, (middle) model reference control, (bottom) optimal control

Problem

- Suppose the trajectory data is available for an unmanned aerial vehicle (UAV) with NN-in-the-loop component, and
- The ability to generate trajectories from the current state (model reference), then
- How do we approximate the reachable sets of the original UAV model in real-time?

Quadrotor Trajectory



Concept

A NN is employed to estimate some unknown dynamics f from time series data trajectories as

$$\dot{\tilde{x}}(t) = \tilde{f}_{\Theta}(\tilde{x}(t), u(t))$$

For initial set $\mathcal{X}_0$ and admissible control set Ω , can we compute:

$$\mathcal{R}^{\tilde{f}}(\tau; \mathcal{X}_0) \triangleq \left\{ \tilde{x}(\tau) \mid \dot{\tilde{x}} = \tilde{f}_{\Theta}(\tilde{x}, u), \tilde{x}(0) \in \mathcal{X}_0, u \in \Omega; \Theta \right\}$$

- Note that NNs are universal approximators for smooth functions over compact sets, so given enough training data $\tilde{f} \approx f$
- Key idea is to excite NN at time step k with some $u_k \in \Omega$, and note data snapshots

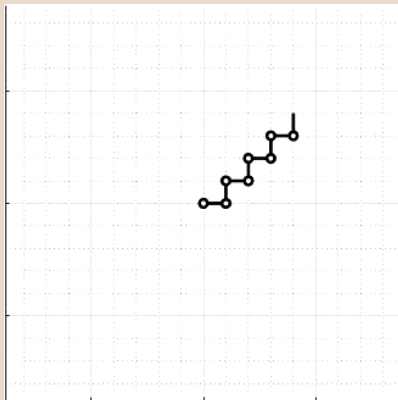
$$\Xi_{k,w} \triangleq \begin{bmatrix} | & & | \\ x_k & \cdots & x_{k+w} \\ | & & | \end{bmatrix}, \Upsilon_{k,w} \triangleq \begin{bmatrix} | & & | \\ u_k & \cdots & u_{k+w} \\ | & & | \end{bmatrix}$$

- Snapshots can be propagated as $\Xi_{k+1,w} = \Gamma_{k,w} \begin{bmatrix} \Xi_{k,w} \\ \Upsilon_{k,w} \end{bmatrix}$, where $\Gamma_{k,w} \triangleq \begin{bmatrix} A & B \end{bmatrix}$

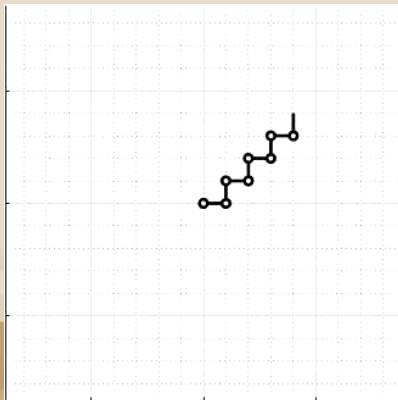
Dynamic Mode Decomposition (DMD) [9]

...but why use DMD to obtain linear approximations to the dynamics? (as opposed to, say, Taylor linearization)

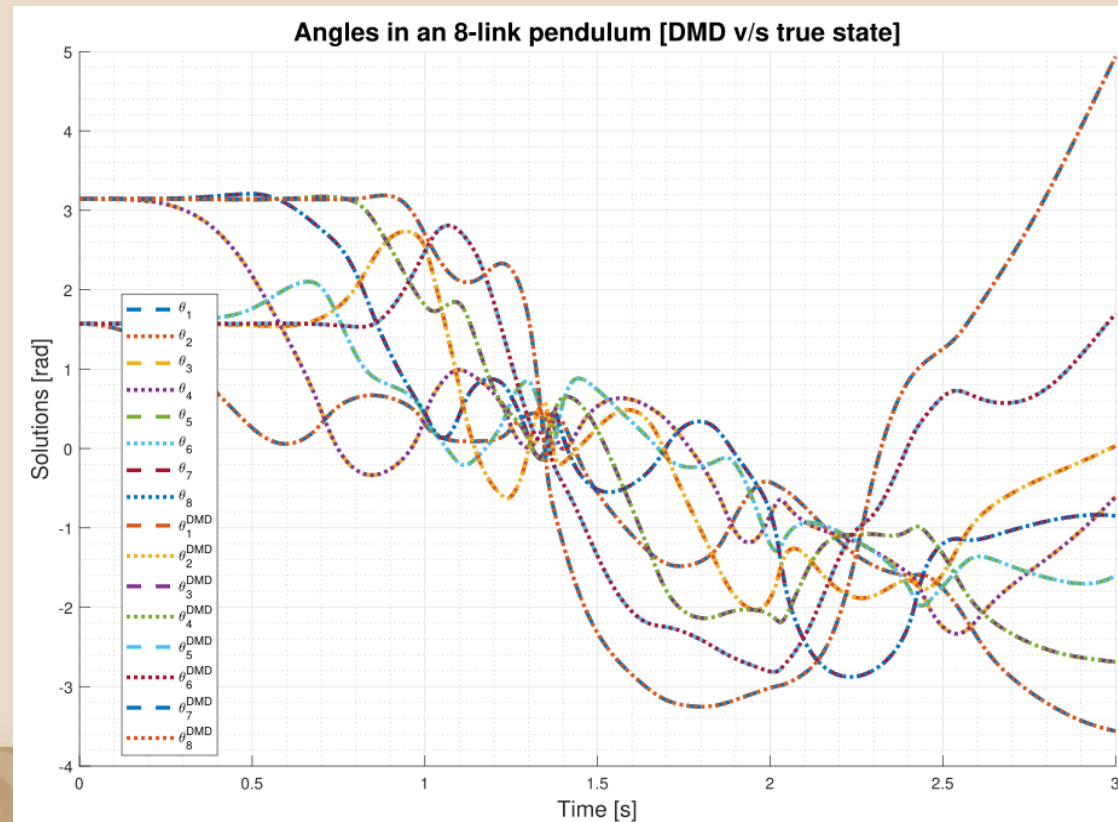
- Because DMD finds best linear approximations given trajectory data, and not pointwise linear fits
- It can be thought of as a finite dimensional approximation of an exact, infinite dimensional, linear lifting, Koopman operator, that maps Ξ_k to Ξ_{k+1}
- Most importantly, it works accurately for many highly nonlinear dynamical models:



← True model



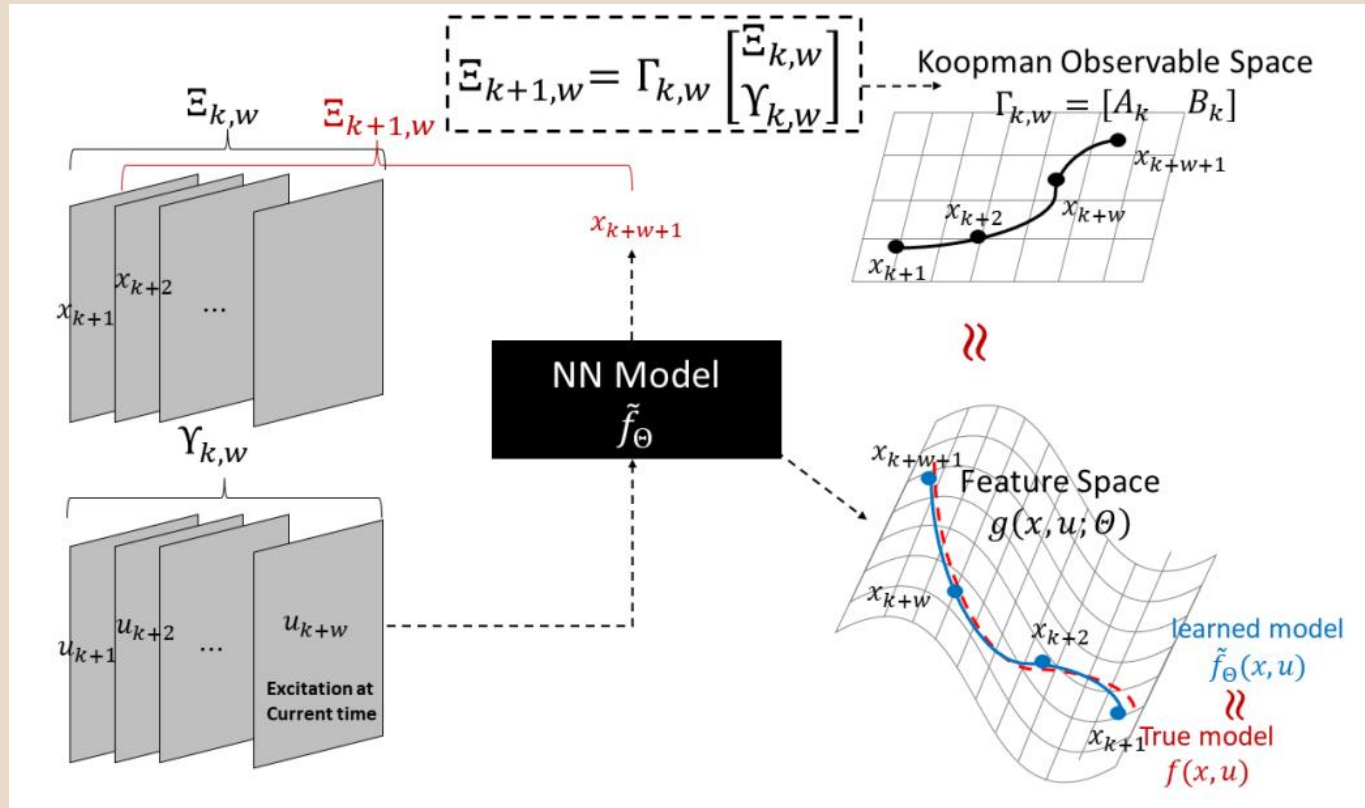
← DMD reconstruction



Solve for A_k, B_k using DMD as:

$$\Gamma_{k,w} = \arg \min \left\| \Xi_{k+1,w} - \Gamma_{k,w} \begin{bmatrix} \Xi_{k,w} \\ \Upsilon_{k,w} \end{bmatrix} \right\|_F, \text{ or}$$

$$\Gamma_{k,w} = [A \quad B] = \Xi_{k+1,w} \begin{bmatrix} \Xi_{k,w} \\ \Upsilon_{k,w} \end{bmatrix}^\dagger$$



From built locally linear model (A_k, B_k) , we can utilize the same polytopic reachable set computation methods from previous section, that is:

- Let hyperplane H_i support the reachable set $\mathcal{R}^{\tilde{f}}$ at some time t
- Admissible control and initial sets are also supported by hyperplanes as:

$$\mathcal{X}_0 = \bigcap_{i=1}^{n_1} \{v \in \mathbb{R}^{n_x} \mid \langle c_i(0), v \rangle \leq \gamma_i(0)\}$$

$$\Omega = \bigcap_{i=1}^{n_2} \{u \in \mathbb{R}^{n_u} \mid \langle d_i, u \rangle \leq \varepsilon_i\}$$

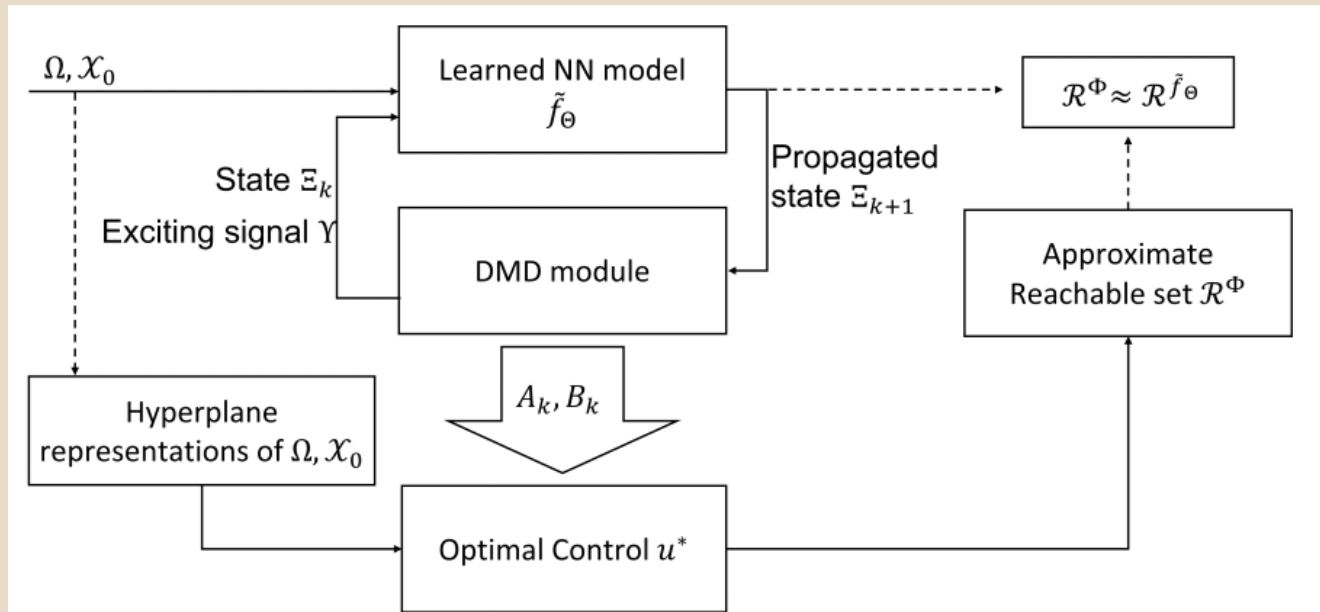
- Let x_i^* be the supporting point of the set at H_i , then polytopic reachable set looks at the propagation of points x_i^* only; Planes $H_i(t, x_i^*)$ support $\mathcal{R}^{\tilde{f}}$ at time t at point x_i^*
- The polytopes provide both under, and over approximation of $\mathcal{R}^{\tilde{f}}$ at all time t as:

$$\text{convexhull}\{x_1^*, \dots, x_{n_1}^*\} \subset \mathcal{R}^{\Phi} \subset \bigcap_1^{n_1} \{\lambda_i, x\} \quad [10, 11]$$

- Solve for optimal control $u_i^*(t)$ that pushes the point $x_i^*(t)$ furthest from away from $H_i \rightarrow$ approximations to

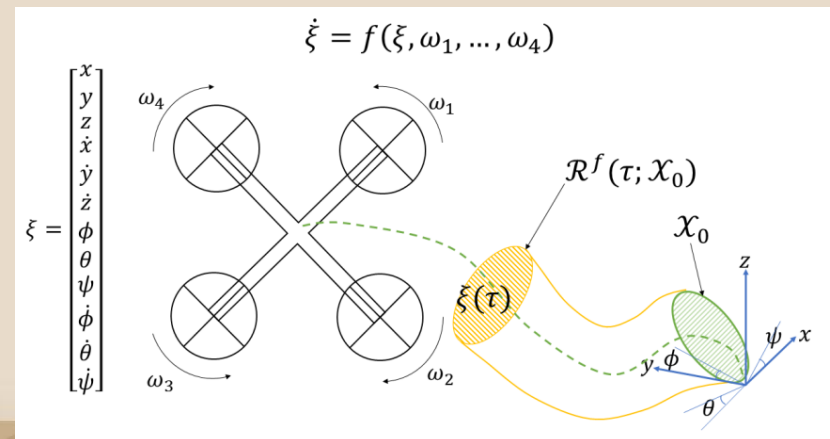
$$\mathcal{R}^{\tilde{f}}(\tau; \mathcal{X}_0) \triangleq \left\{ \tilde{x}(\tau) \mid \dot{\tilde{x}} = \tilde{f}_{\Theta}(\tilde{x}, u), \tilde{x}(0) \in \mathcal{X}_0, u \in \Omega; \Theta \right\}$$

The overall method can be summarized as:



We will consider a realistic example of a fully nonlinear, 12 DOF quadrotor model where:

- Compute $\mathcal{R}^{\tilde{f}}$ for NN model
- Compute $\mathcal{R}^{\tilde{f}}$ for parameter variation



- 12 DOF Quadrotor dynamics [12] :

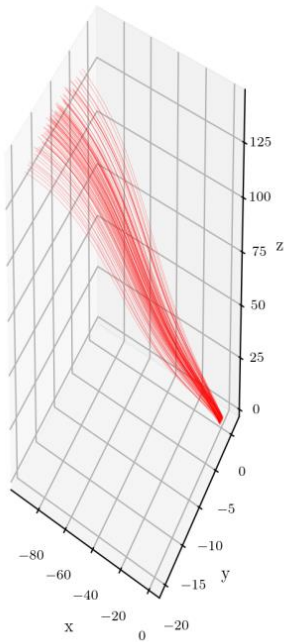
$$\begin{bmatrix} \ddot{x} \\ \ddot{y} \\ \ddot{z} \end{bmatrix} = \begin{bmatrix} -\frac{u_1}{m} (\sin \phi \cos \psi + \cos \phi \cos \psi \sin \theta) \\ -\frac{u_1}{m} (\cos \phi \sin \psi \sin \theta - \cos \psi \sin \psi) \\ g - \frac{u_1}{m} (\cos \phi \cos \theta) \end{bmatrix}$$

$$\begin{bmatrix} I_{xx} \ddot{\phi} \\ I_{yy} \ddot{\theta} \\ I_{zz} \ddot{\psi} \end{bmatrix} = \begin{bmatrix} u_2 \\ u_3 \\ u_4 \end{bmatrix} - \begin{bmatrix} (I_{zz} - I_{yy}) \dot{\theta} \dot{\psi} \\ (I_{xx} - I_{zz}) \dot{\phi} \dot{\psi} \\ (I_{yy} - I_{xx}) \dot{\theta} \dot{\phi} \end{bmatrix}$$

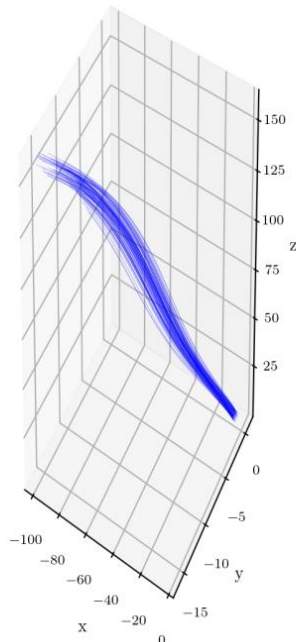
$$\begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \end{bmatrix} = \begin{bmatrix} k_f & k_f & k_f & k_f \\ -lk_f & lk_f & lk_f & -lk_f \\ lk_f & lk_f & -lk_f & -lk_f \\ k_m & -k_m & k_m & -k_m \end{bmatrix} \begin{bmatrix} \omega_1^2 \\ \omega_2^2 \\ \omega_3^2 \\ \omega_4^2 \end{bmatrix}$$

- Initial set is simply a hypercube of side 0.5 centered at the origin
- Admissible control set: $\omega_i(t) = v_i(t) + w_i, w_i(t) \sim \mathcal{U}_{[-0.25, 0.25]}$ for $i = 1, \dots, 4$
- Nonlinear dynamics are learned with a causal-NN with 3 layers is trained over a RMSE loss function with 100 trajectories, over 2000 epochs
- Total training time on an Intel Xeon CPU running at 2.20 GHz, with 13GB memory and 56MB cache size was approximately 80 seconds
- Trained NN is excited with a randomly chosen admissible RPM input to each quadrotor

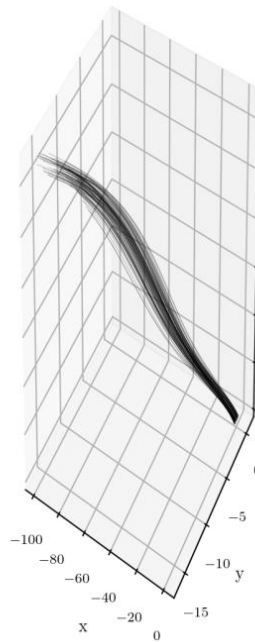
True Trajectory



Learned Trajectory

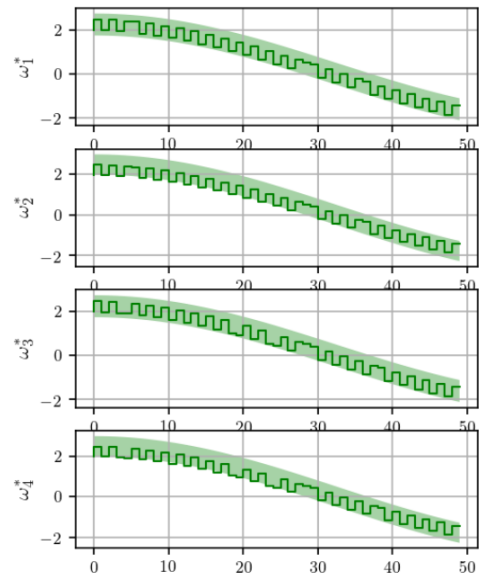
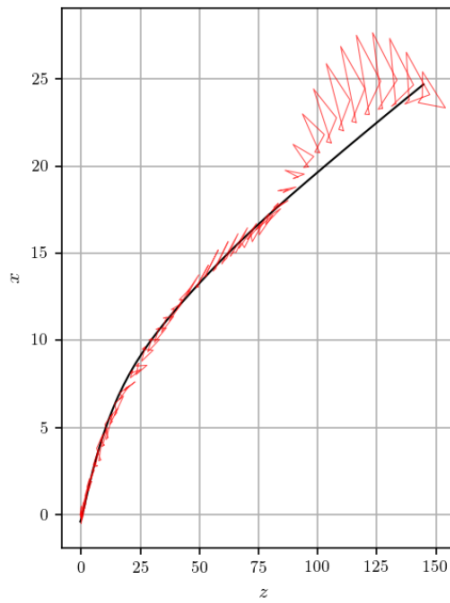
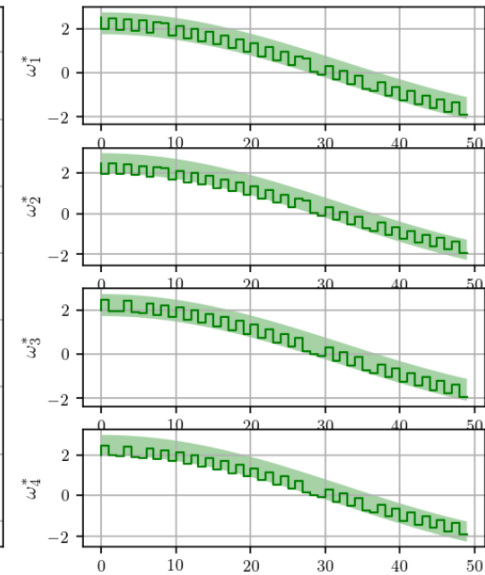
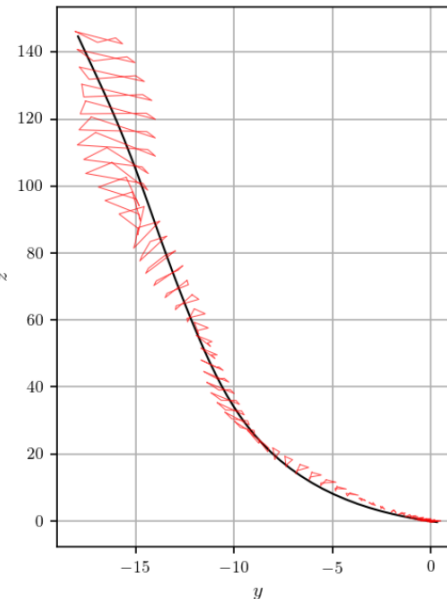


DMD Trajectory

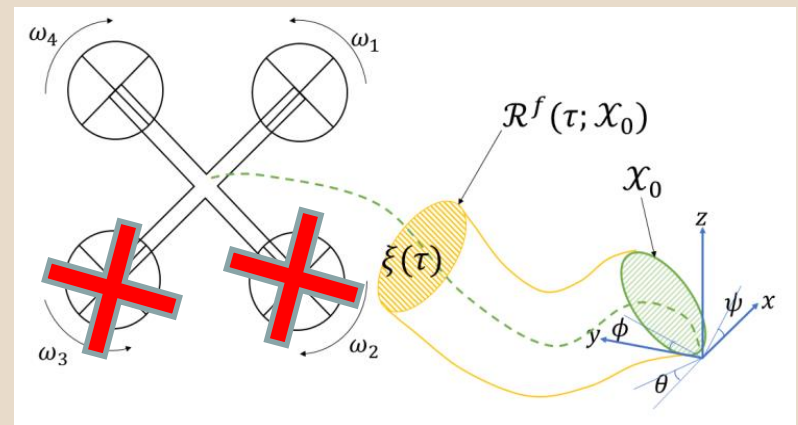


NN and DMD estimation accuracies

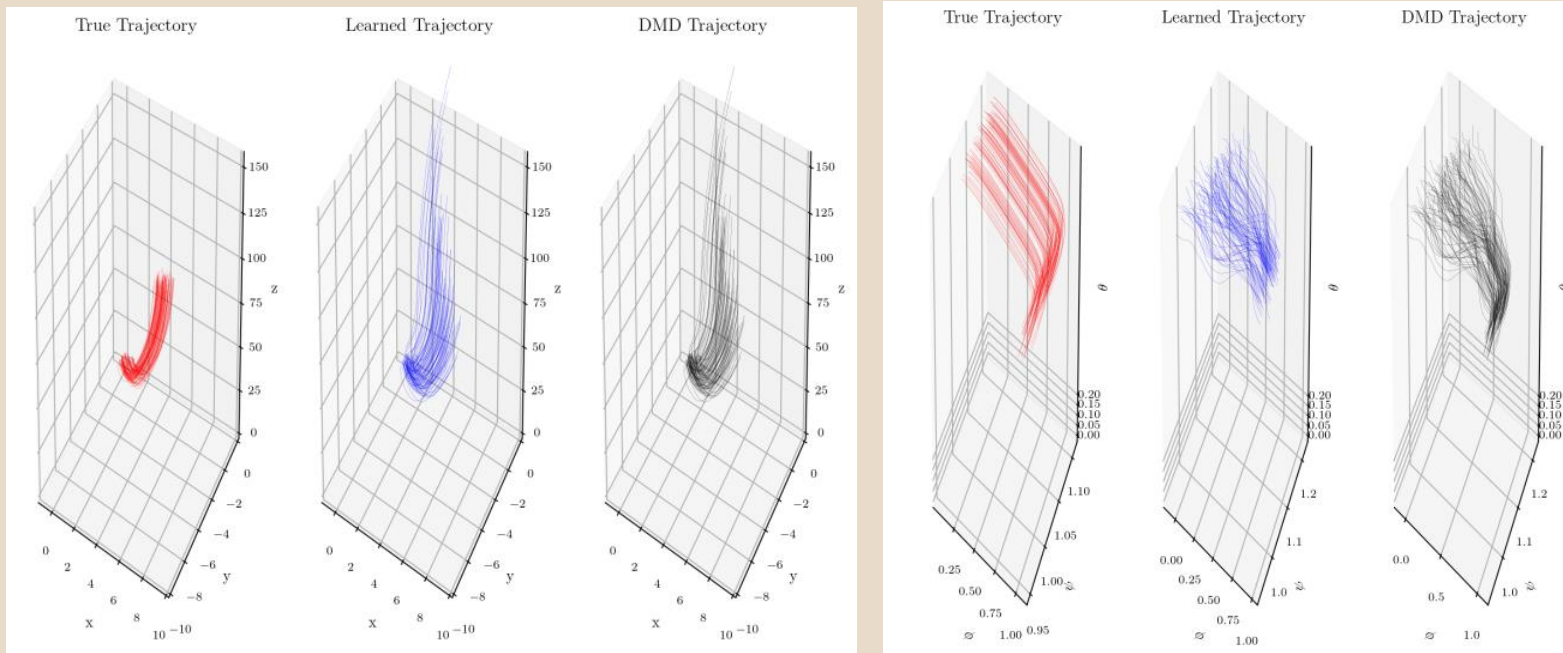
(red-black) Inner approximation of Reachable sets in y, z and x, z directions, (green) admissible control set



Reachable set under Rotor Failure: *Failure at Rotor 2 & 3*

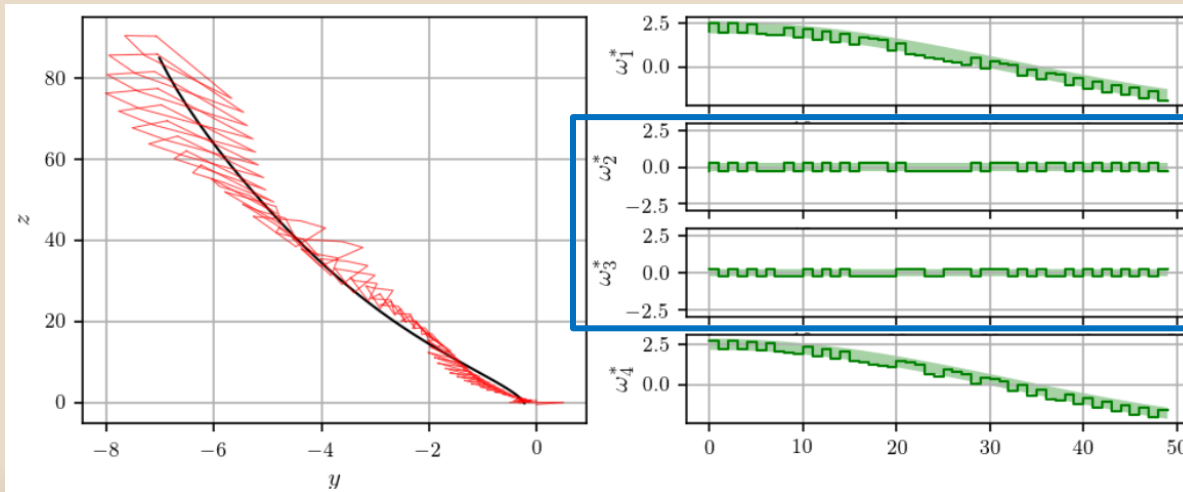
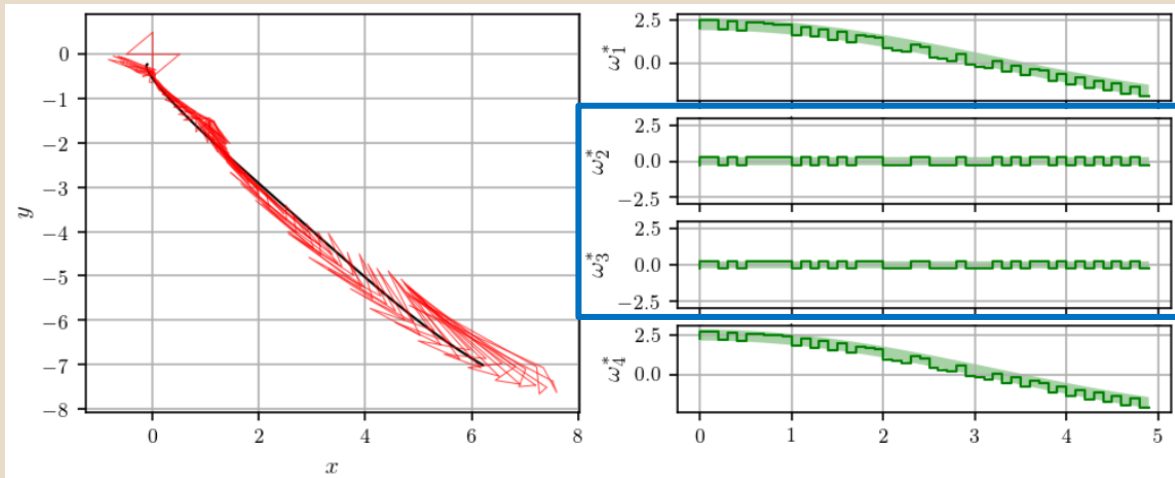


NN and DMD estimation accuracies under rotor failure



(left) 3D State trajectories in x, y, z (right) 3D Attitude trajectories

Reachable set under Rotor Failure: *Failure at Rotor 2 & 3*



- Proposed method still estimates reachable sets under rotor failure
- No MILP solutions required
- Utilizes only data to construct (A_k, B_k) equivalent to the NN
- Additional computational cost is only $\mathcal{O}(\exp[\cdot])$ to propagate the state
 - This can be done efficiently using linear algebra libraries
- Additional computation time was < 1 seconds to compute $\mathcal{R}^{\tilde{f}}$ at sampling rate $\Delta t = 0.1s$
- Scales well with high dimensional (12DOF) dynamics
- Able to address nonlinear dynamics, including parameter uncertainties

Outline

- Background and Motivation
- Research Gaps
- Research Objectives
- Post-Prelim Results
 - Distributed Reachable Set Computation for Multi-Agent Systems
 - Data-Driven Reachable set Computation for Neural Network Models
 - **Data-Driven Cyberattack against Multi-Agent Systems**
- Conclusion

Data-Driven Cyberattack against Network Control Systems

Motivation

- Network control systems allow control engineers to solve complex tasks, design sophisticated control schemes, and model cooperation of spatially separate entities, via data sharing across communication networks
 - → NCSs find applications in distributed control, power grids, multi-robot cooperation, etc.
- Increased reliance on communication → NCSs often communicate over insecure channels + susceptible to cybersecurity threats.
 - E.g., StuxNet, Capturing of RQ-170 recon a/c, cyber threats to autonomous driving

Scope

- Smart attackers can eavesdrop to observe/measure NCS data → construct auxiliary NCS models + identify underlying communication graphs → perform realistic hybrid attacks (mixture of two different cyberattacks)

Data-driven cyberattack synthesis methodology to perform combinations of gray box attacks against NCSs to cause safety violations, with incomplete system knowledge

- Consider an NCS (e.g., LTI unmanned aerial vehicles) with coupled state feedback control laws as:

$$\begin{aligned}\dot{x}_i(t) &= A_i x_i(t) + B_i u_i(t), \\ u_i(t) &= K_{ii} x_i(t) + \sum_{j \in \mathcal{N}_i(t)} K_{ij} (x_i(t) - x_j(t))\end{aligned}$$

$\equiv$ from attacker's perspective

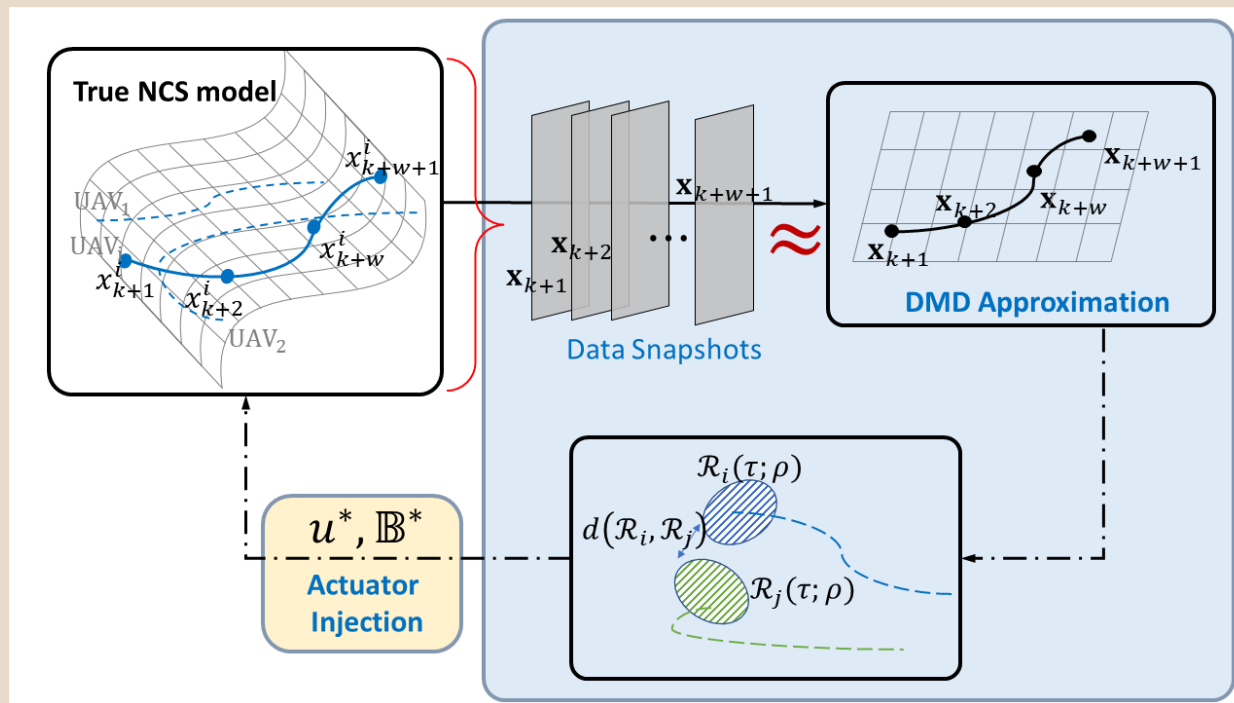
$$\begin{aligned}\dot{\mathbf{x}}(t) &\triangleq \mathbb{A}_{\mathcal{G}(t)} \mathbf{x}(t) \\ \mathbf{x}(t) &\triangleq [x_1(t)^T, x_2(t)^T, \dots, x_N(t)^T]^T \\ \mathbb{A}_{\mathcal{G}(t)} &\triangleq \text{diag} \{A_i + B_i K_{ii}\}_{i \in \mathcal{V}} + B_i K_{ij} \otimes \mathcal{L}_{\mathcal{G}(t)}\end{aligned}$$

- where the attacker is observing states $\mathbf{x}(t)$ at some time instant (by eavesdropping, or state estimation)
- The false data injection attack can be written as the attack synthesis problem below:

$$\begin{aligned}&\text{find } \mathbb{B}^a, \mathbf{u}^a(t) \\ &\text{such that } \dot{\mathbf{x}}(t) = \mathbb{A}_{\mathcal{G}(t)} \mathbf{x}(t) + \mathbb{B}^a \mathbf{u}^a(t), \\ &\quad \exists t^* \geq t_1 \text{ where } \|x_i(t^*) - x_j(t^*)\|_2 \geq d^*, \\ &\quad \|\mathbf{u}^a(t)\|_2 \leq \rho \text{ for all } t \geq t_1\end{aligned}$$

- where ρ is the attacker's budget, and the attacker is unaware of $\{A_i(t), B_i(t), \{K_{ij}\}_{j \in \mathcal{N}_i(t)}\}_{i \in \mathcal{V}}$, but can collect the discrete-time trajectory data $\{x_1(t), \dots, x_N(t)\}_{t=t_0}^{t=t_1}$ over time interval $[t_0, t_1]$

- If the trajectory data could be used to construct approximate models using DMD, the injection vehicles can be chosen according to pairs $i - j$ for which the reach sets $\mathcal{R}(t_1; t_0)$ are the closest:

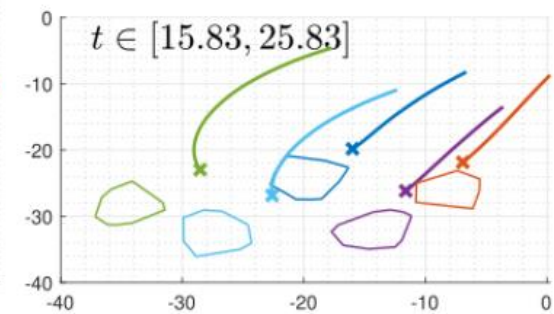
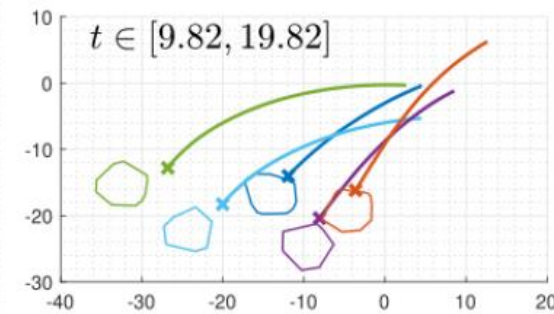
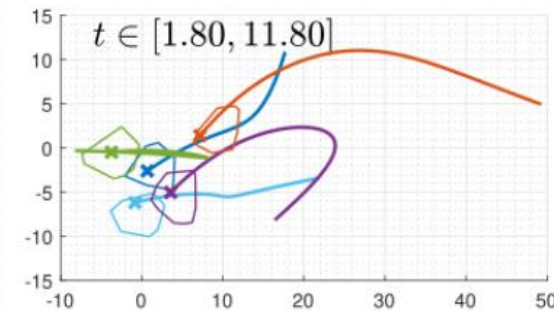
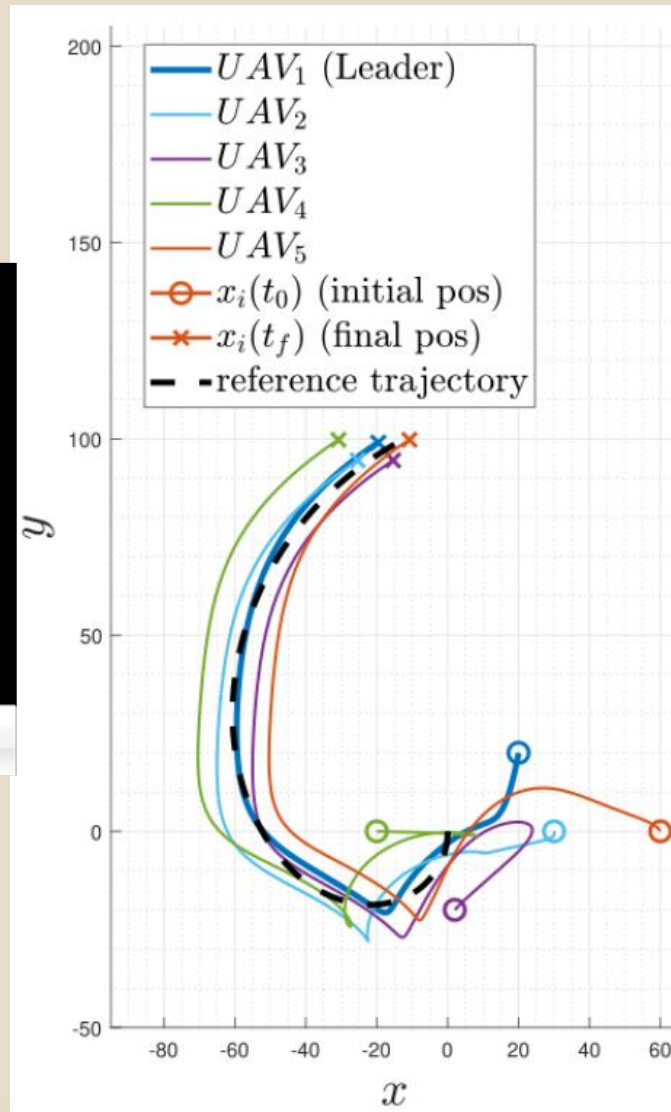
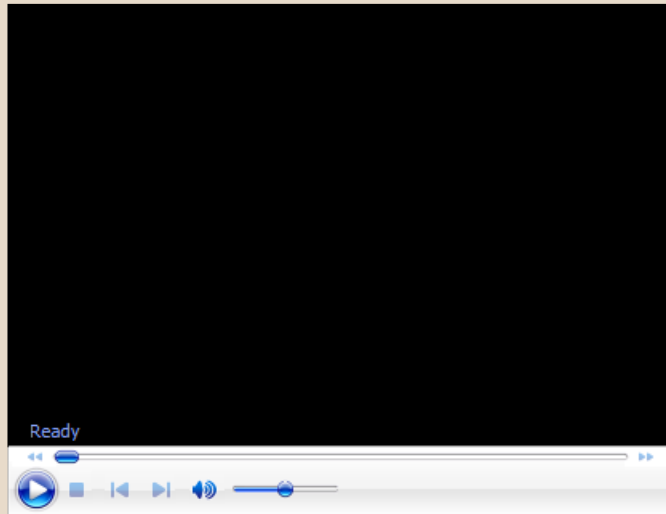
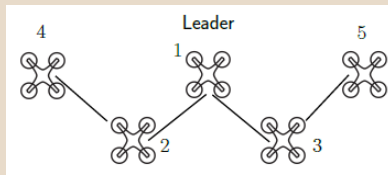


$$(i^*, j^*) = \arg \max_{i, j} d(\mathcal{R}_i(\tau; \rho), \mathcal{R}_j(\tau; \rho))$$

- That is,

$$\mathbf{u}^a = \arg \max_{\|\mathbf{u}\| \leq 0.05} d(\mathcal{R}_i(\tau + \Delta t; \rho), \mathcal{R}_j(\tau + \Delta t; \rho))$$

Let us look at the effect of such an FDI attack on an NCS with 5 quadrotors performing formation control



Attacker injects FDI by solving (P1)

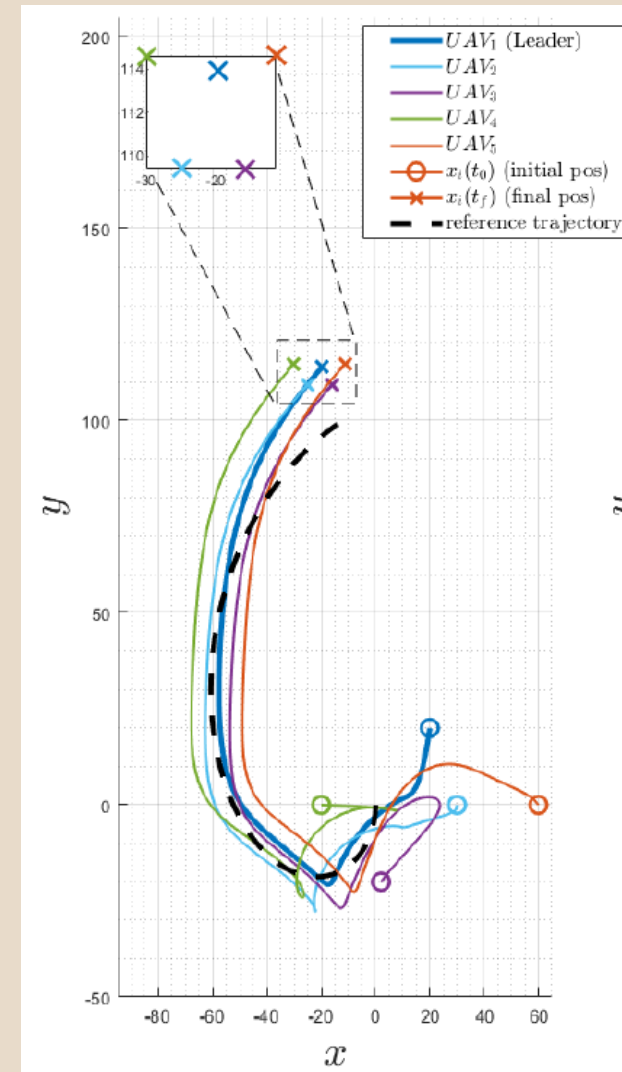
Problem P1 to perform FDI of strength ρ

- Find vulnerable agent pair that can be deviated maximally:

$$(i^*, j^*) = \arg \max_{i,j} d(\mathcal{R}_i(\tau; \rho), \mathcal{R}_j(\tau; \rho))$$

$$\mathbf{u}^a = \arg \max_{\|\mathbf{u}\| \leq 0.05} d(\mathcal{R}_i(\tau + \Delta t; \rho), \mathcal{R}_j(\tau + \Delta t; \rho))$$

- The FDI results in:
 - UAVs no longer stick to the reference trajectory ($\rightarrow$ FDI has introduced a bias in reference tracking), but
 - 'W'-shaped formation still being preserved



Attacker injects DoS, then FDI by solving (P2)

- P2: A smarter attacker can accompany the FDI of same budget with a preceding DoS attack (unsophisticated to carry out)
 - DMD-based approximation of the NCS already computed by eavesdropping + snapshot data
 - Underlying graph (Laplacian) $\mathcal{L}_G$ can be approximated by finding the matrix $\hat{\mathcal{L}}$ that comes closest to the Laplacian
 - Resulting matrix finding problem is:

$$\hat{\mathcal{L}} = \arg \min_L \|K - (S + T \otimes L)\|_F \quad \longleftrightarrow \quad \mathbb{A}_{\mathcal{G}(t)} \triangleq \text{diag} \{A_i + B_i K_{ii}\}_{i \in \mathcal{V}} + B_i K_{ij} \otimes \mathcal{L}_{\mathcal{G}(t)}$$

- But note the Laplacian has properties: $L \succeq 0, L\mathbf{1} = 0$
- Problem above can then be written as a ‘quasi’ SDP*:

$$\begin{aligned} & \min \gamma \\ & \text{subject to} \quad \begin{bmatrix} \gamma I & K - (S + T \otimes L) \\ [K - (S + T \otimes L)]^T & \gamma I \end{bmatrix} \succ 0 \\ & L \succeq 0, L\mathbf{1} = 0 \end{aligned}$$

$\lambda_2(\hat{\mathcal{L}}) \rightarrow$ solves sparsest cut problem for $\mathcal{G} \rightarrow$ DoS-ing edge corresponding to λ_2 have maximal impact on graph connectivity

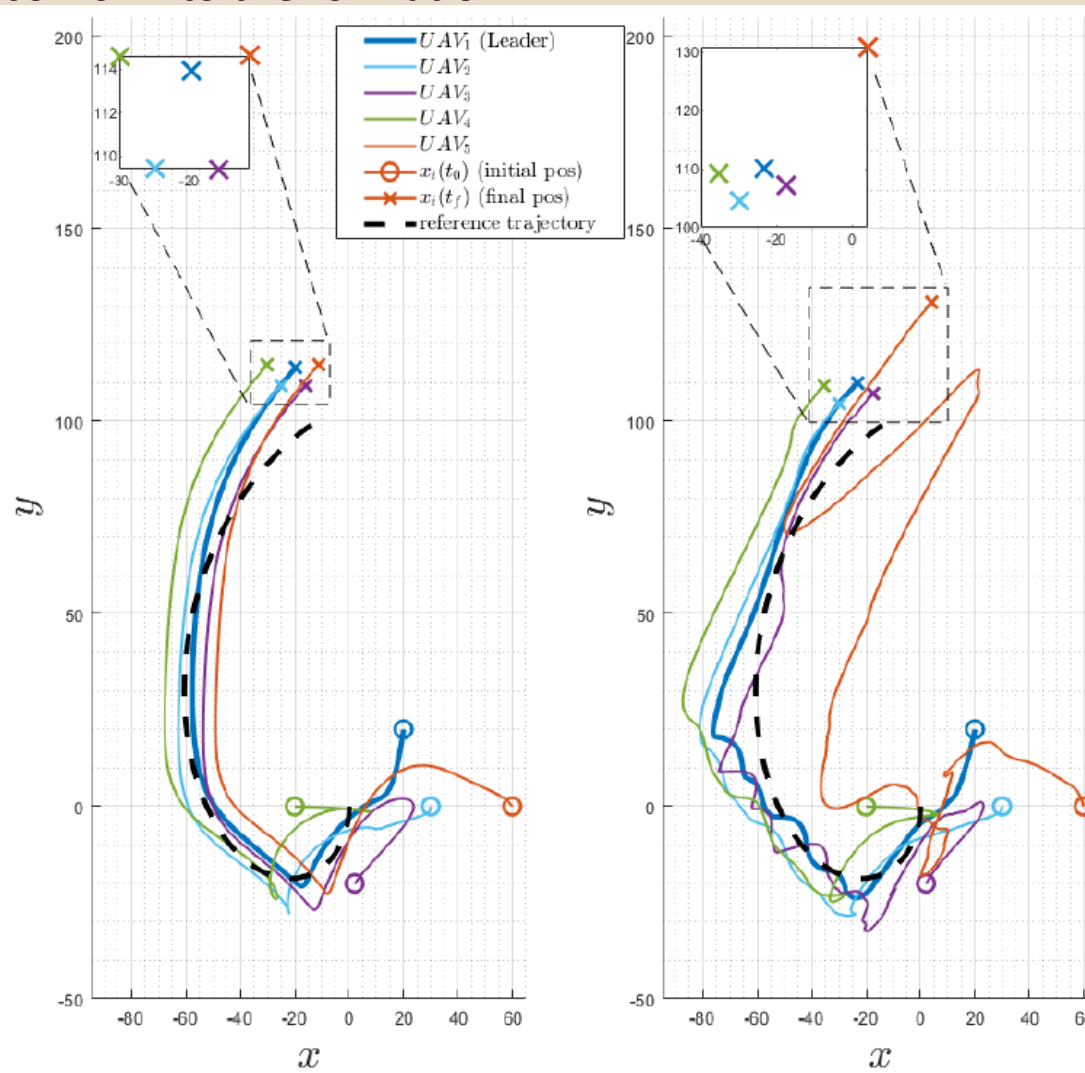
Compute $\lambda_2(\hat{\mathcal{L}}) \rightarrow$ vulnerable edge to DoS $\rightarrow$ FDI attack

*proper SDP is obtained if T is fixed; this is a ‘feasibility’ SDP where S, T are free variables; Quasi-SDP is solved by iterative SDP with random initial conditions

Smart attack: *FDI + DoS attack using trajectory data*

Attacker can worsen performance very easily by preceding the FDI with a DoS attack →

- Not only is the reference trajectory following worsened, but UAV corresponding to $\lambda_2(\cdot)$ is unable to conform to the formation

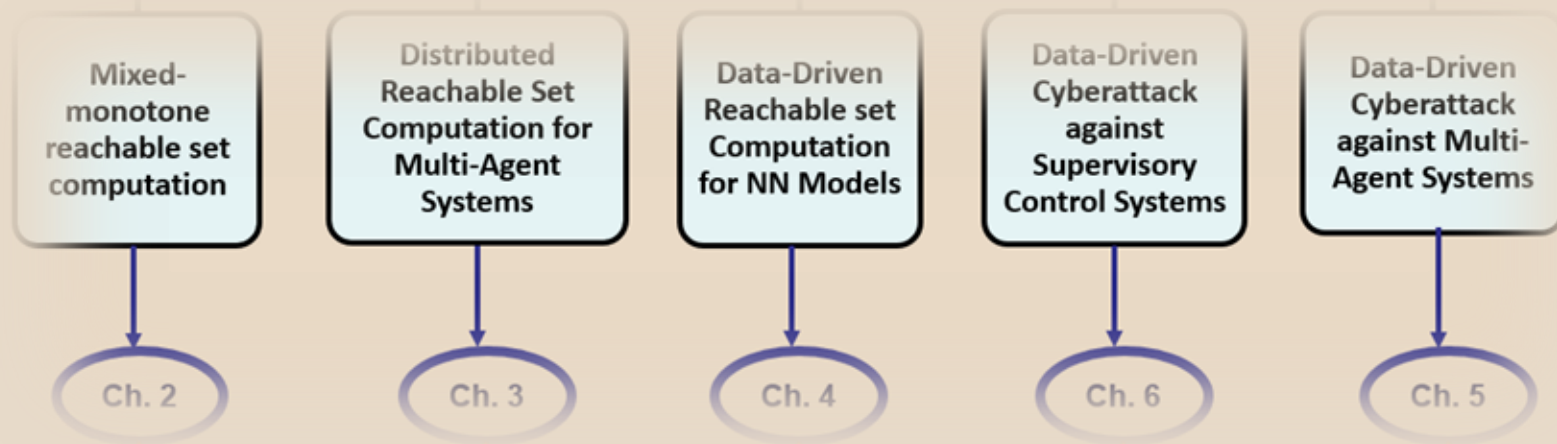


Conclusion

Contributions

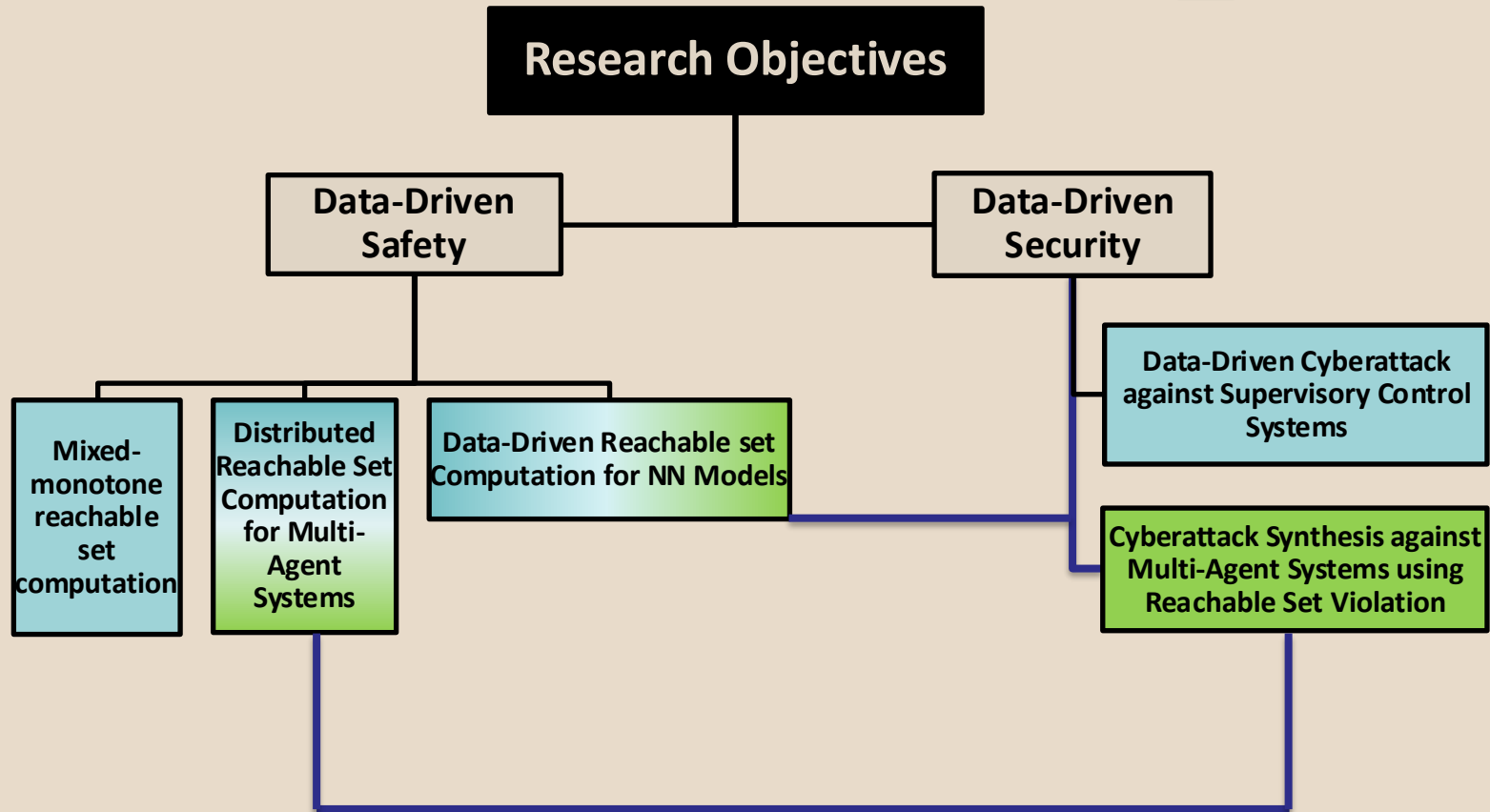
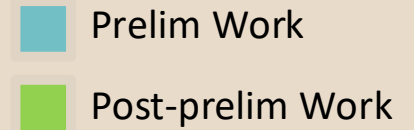
- While numerical methods exist to compute reach sets, Chapter 2 accomplishes the same with an advantage of real-time applicability, while handling unknown system dynamics,
 - ...at the expense of rapid, but loose over-approximations
- Chapter 3 allows for computing approximate reachable sets for NNs in real-time is very much an open problem, and the results can be far reaching as NNs are used in all parts of control design
 - ...however, accuracy relies on the accuracy of the trained NN → asymptotic accuracy with # data points
- And offers a systematic way to compute bounds on NN outputs in real-time, while respecting dynamical data → model + data-driven approximate reachability
- Chapter 3 was extended in Chapter 4 to apply to network control systems, thereby overcoming curse of dimensionality of the order $\mathcal{O}(\exp[n \times N])$
 - ...at the expense of a distributed optimization step, and convergence holding for specific graph structures
- Chapters 5 & 6 present realistic, grey box cyberattack vulnerabilities of NCSs and supervisory control systems, where attackers can cause more harm to CPSs by employing multiple easily to conduct, inexpensive attacks in conjunction
 - ...at the expense of being data-hungry methods

Recommendations for Future Work



- Computation of flight envelopes/operating capabilities of UAVs in real-time
 - Methods in Chapters 2 & 3 can be effectively utilized
- Convergence of the methods in Chapters 3 and 5 can be discussed over stochastic graphs
 - Techniques from stochastic distributed optimization can be utilized
- Certification of approximation levels for over/under-approximations in Chapters 2 and 3
 - Works by Kurzchansky et. al. can be utilized to certify approximation levels of reachable sets [16]
- Extensions of methods in Chapters 2 through 4 for discrete-event systems
- Hybrid attacks in Chapters 5 and 6 only considered FDI and DoS attacks; smarter attackers can perform stealthy attacks of other kinds, e.g., man-in-the-middle attacks
- Mixed monotone schemes from Chapter 2 can be extended to hybrid dynamical systems
 - A two-level system identification can be performed to identify polynomial dynamics over guard conditions that also lie on polynomial manifolds → mode-matched mixed monotone reachable set over-approximation

Research Map



References

- [1] Fathian, Kaveh, Tyler H. Summers, and Nicholas R. Gans. "Distributed formation control and navigation of fixed-wing UAVs at constant altitude." 2018 International Conference on Unmanned Aircraft Systems (ICUAS). IEEE, 2018.
- [2] Hunt, Kenneth J., D. Sbarbaro, R. Żbikowski, and Peter J. Gawthrop. "Neural networks for control systems—a survey." *Automatica* 28, no. 6 (1992): 1083-1112.
- [3] K. Fathian, T. H. Summers, N. R. Gans, "Robust Distributed Formation Control of Agents with Higher-Order Dynamics," IEEE Control Systems Letters, 2018
- [4] Song, Yang, Hao Yu, Sai Manoj Pudukotai Dinakarrao, and Guoyong Shi. "SRAM dynamic stability verification by reachability analysis with consideration of threshold voltage variation." In *Proceedings of the 2013 ACM International symposium on Physical Design*, pp. 43-49. 2013.
- [5] Uma, M. and Padmavathi, G., 2013. A Survey on Various Cyber Attacks and their Classification. *Int. J. Netw. Secur.*, 15(5), pp.390-396.
- [6] Singh, S. and Silakari, S., 2009. A survey of cyber attack detection systems. *International Journal of Computer Science and Network Security*, 9(5), pp.1-10.
- [7] Coogan, S., 2020, December. Mixed monotonicity for reachability and safety in dynamical systems. In *2020 59th IEEE Conference on Decision and Control (CDC)* (pp. 5074-5085). IEEE.
- [8] Coogan, S. and Arcak, M., 2015, April. Efficient finite abstraction of mixed monotone systems. In *Proceedings of the 18th International Conference on Hybrid Systems: Computation and Control* (pp. 58-67).
- [9] Mauroy, A., Mezić, I. and Susuki, Y. eds., 2020. *The Koopman Operator in Systems and Control: Concepts, Methodologies, and Applications* (Vol. 484). Springer Nature.
- [10] Varaiya, P., 2000. Reach set computation using optimal control. In *Verification of Digital and Hybrid Systems* (pp. 323-331). Springer, Berlin, Heidelberg.
- [11] Hwang, I., Stipanović, D.M. and Tomlin, C.J., 2005. Polytopic approximations of reachable sets applied to linear dynamic games and a class of nonlinear systems. In *Advances in control, communication networks, and transportation systems* (pp. 3-19). Birkhäuser Boston.
- [12] Wang, P., Man, Z., Cao, Z., Zheng, J. and Zhao, Y., 2016, November. Dynamics modelling and linear control of quadcopter. In *2016 International Conference on Advanced Mechatronic Systems (ICAMEchS)* (pp. 498-503). IEEE.
- [13] Mou, S., Liu, J. and Morse, A.S., 2015. A distributed algorithm for solving a linear algebraic equation. *IEEE Transactions on Automatic Control*, 60(11), pp.2863-2878.
- [14] Goodfellow, I., Bengio, Y. and Courville, A., 2016. *Deep learning*. MIT press.
- [15] Desai, J.P., Ostrowski, J. and Kumar, V., 1998, May. Controlling formations of multiple mobile robots. In *Proceedings. 1998 IEEE International Conference on Robotics and Automation (Cat. No. 98CH36146)* (Vol. 4, pp. 2864-2869). IEEE.
- [16] Kurzhanski, Alexander B., and Pravin Varaiya. "On ellipsoidal techniques for reachability analysis. Part I: external approximations." *Optimization methods and software* 17.2 (2002): 177-206.

Thank you

PURDUE

Acknowledgment and Thanks

- Committee Members: Prof. Inseok Hwang, Prof. Arthur Frazho, Prof. Dengfeng Sun, Prof. Shaoshuai Mou
- Labmates, and friends, especially Ridhi
- Family members, especially my mother!

List of Publications

Journal Publications

- **Thapliyal, O.** and Hwang, I., 2023. Approximating Reachable Sets for Neural Network-Based Models in Real Time via Optimal Control. *IEEE transactions on control systems technology*.
- **Thapliyal, O.**, & Hwang, I. (2022). Approximate Reachability for Koopman Systems Using Mixed Monotonicity. *IEEE Access*, 10, 84754-84760
- [under review] Clarke, S., Hwang, S., **Thapliyal, O.** and Hwang, I., 2022. Distributed Denial-of-Service Resilient Control for Urban Air Mobility Applications. *AIAA Journal of Aerospace Information Systems*.
- [under preparation] **Thapliyal, O.**, Clarke, S., Hwang, S., and Hwang, I., 2023. Distributed Computation of Reachable Sets for Multi-Agent Systems. *IEEE transactions on control of network systems*.
- **Thapliyal, O.**, Nandiganahalli, J.S. and Hwang, I., 2019. Kalman filtering with state-dependent packet losses. *IET Control Theory & Applications*, 13(2), pp.306-312.
- Deshmukh, R., **Thapliyal, O.**, Kwon, C. and Hwang, I., 2018. Distributed state estimation for a stochastic linear hybrid system over a sensor network. *IET Control Theory & Applications*, 12(10), pp.1456-1464.

Conference Publications

- [under review] **Thapliyal, O.**, & Hwang, I. (2022). Data-driven Cyberattack Synthesis against Network Control Systems. arXiv preprint arXiv:2211.05203 *IFAC World Congress 2023*.
- Clarke, S., **Thapliyal, O.**, Hwang, S., and Hwang, I. 2021. Attack-Resilient Distributed Optimization-based Control of Multi-Agent Systems with Dual Interaction Networks. *AIAA Scitech 2021 Forum*.
- **Thapliyal, O.** and Hwang, I., 2021, June. Learning Based Cyberattack Design and Defense for Supervisory Control Systems. In *2021 European Control Conference (ECC)*. IEEE.
- **Thapliyal, O.** and Hwang, I., 2021, May. Path Planning for a Network of Robots with Distributed Multi-Objective Linear Programming. In *2021 American Control Conference (ACC)* (pp. 4643-4648). IEEE.
- Shethia, S., Gupta, A., **Thapliyal, O.**, & Hwang, I. (2021). Distributed Fast-Tracking Alternating Direction Method of Multipliers (ADMM) Algorithm with Optimal Convergence Rate. *2021 IEEE International Conference on Systems, Man, and Cybernetics (SMC)* (pp. 976-981).
- Sivaramakrishnan, V., **Thapliyal, O.**, Vinod, A., Oishi, M. and Hwang, I., 2019, December. Predicting Mode Confusion Through Mixed Integer Linear Programming. In *2019 IEEE 58th Conference on Decision and Control (CDC)* (pp. 2442-2448). IEEE.
- **Thapliyal, O.**, Nandiganahalli, J.S. and Hwang, I., 2017, December. Optimal state estimation for LTI systems with imperfect observations. In *2017 IEEE 56th Annual Conference on Decision and Control (CDC)* (pp. 2795-2800). IEEE.

Appendix

[Extra Slides]

Contents

1. Data-Driven Cyberattack against Supervisory Control Systems
2. Mixed Monotone Reachability for Data-Driven System ID
3. Polytopic Reachability: Optimal Control Problem
4. Extending Distributed Reachable Set Computation: $\mathbb{A} \rightarrow \mathbb{A}(t)$
5. Basics NNs
6. Basics GANs

Data-Driven Cyberattack against Supervisory Control Systems

Motivation

- Supervisory control finds applications in human-machine interactions, networked control systems, automatic logic, hybrid systems, etc.
- Addresses discrete logic + continuous dynamical models
 - Discrete + continuous interplay $\Rightarrow$ supervisory controllers have vulnerabilities that can be exploited by cyberattacks

Scope

- Hybrid controllers can bypass the vulnerabilities of linear systems by switching among usable controllers when subjected to cyberattacks ^[1, 2]
- Discrete logic-based vulnerabilities can be exploited, even by 'gray box' cyberattackers

Learning based methodology to perform gray box cyberattacks on supervisory controllers where the attacker has incomplete system knowledge, and Learning based defense methodology against cyberattacks of unknown synthesis mechanisms

- Consider a cyberphysical system (CPS) subject to supervisory control with discrete logic

$$x_{k+1} = Ax_k + Bu_k(\sigma_k)$$

- The supervisor logic is given by a finite automaton to model rules for the supervisory controller as

$$S = (\Sigma, \mathcal{X}, \mathcal{U}, \delta, \phi)$$

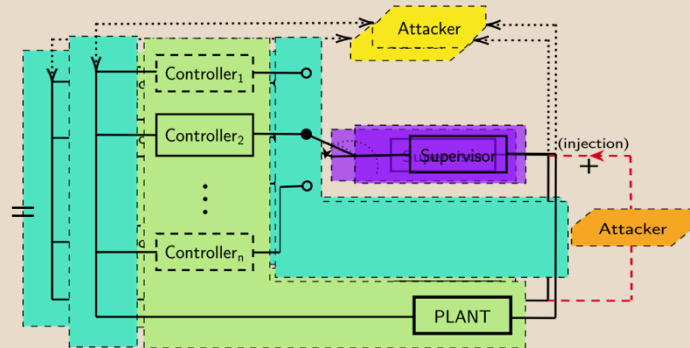
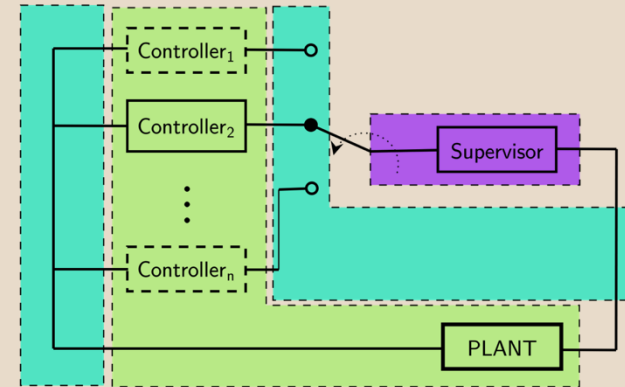
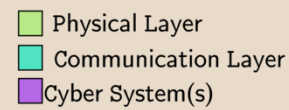
$$\sigma_k = \delta(\sigma_{k-1}, x_k), \text{ and } u_k = \phi(\sigma_k)$$

- E.g., a supervisor that switches control input among a bank of pure state-feedback controllers, where each controller operates in a partition of the state space, is given as

$$\sigma_k = \delta(\sigma_{k-1}, x_k) = j \text{ if } \mathcal{G}_{\sigma_{k-1}j}(x_k) \geq 0$$

$$u_k(j) = K_j x_k, \text{ for gains indexed by } j \in \Sigma$$

- Controller discrete states σ_k allowed to transition through guard conditions $\mathcal{G}_{ij}$, determining the boundary between $\sigma_k = i$ and $\sigma_k = j$
- The transition map can be visualized as the boundary of partition of the state space where controller i is operable



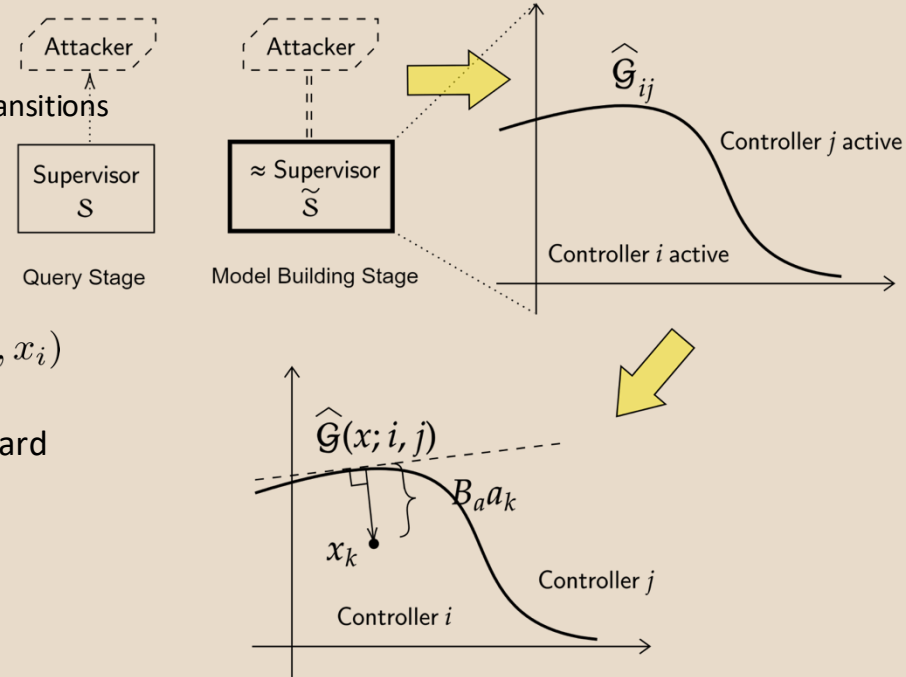
Attacker gathering information
Attacker injecting data

Attack Problem

- The attacker has partial knowledge about the CPS
 - $\therefore$ attacker first needs to develop a model of discrete state transitions
 - In this work, this is posed as a learning problem
- Query data is collected through observing open-loop supervisor behavior data $\mathcal{D} = \{(y_i, x_i)\}$, $y_i = \delta(\sigma_i, x_i)$
- Finding the discrete state transition is posed as a standard learning problem [14]:

$$J(\Theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^M \left[y_i^{(j)} \log (h_{\Theta}(x_i))^{(j)} + (1 - y_i^{(j)}) \log (h_{\Theta}(x_i))^{(j)} \right] + \frac{\lambda}{2N} \sum_{l=1}^L \|\Theta\|^2$$

- This gives the attacker an estimate of the supervisor's discrete state mapping $\hat{\delta}_{\Theta^*}$
- The data-driven attack problem is aimed at causing mode confusion in the estimate $\hat{\delta}_{\Theta^*}$ [14]



$$\begin{aligned} & \text{find } a_k, \|a_k\|_{\infty} \leq \rho \\ & \text{such that } \hat{\delta}(\sigma_{k-1}, x_k; \Theta^*) \neq \hat{\delta}(\sigma_{k-1}, x_k + a_k; \Theta^*) \\ & \text{given: } \Sigma, \mathcal{X}, \mathcal{U} \end{aligned}$$

Defense Mechanism

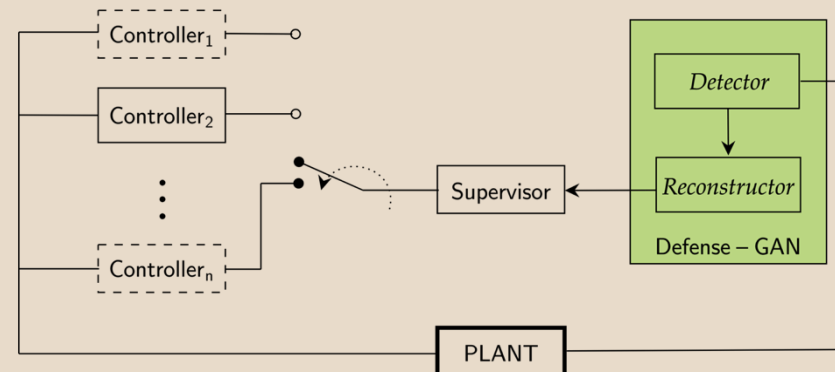
Defense Generative Adversarial Network (GAN)

- A Realistic defense against cyberattacks must be agnostic to attack generation mechanisms and related distributions
- A successful defense must:
 - predict whether the supervisory control system has been compromised, and
 - reconstruct the compromised signal to the supervisor
- A distribution agnostic method for this is based on designing a generative adversarial network (GAN) [6]
- Defense GAN is composed of a *detector* to predict the occurrence of an attack, and a *reconstructor* to improve compromised supervisor output

- Detector trained to detect probability of an undesired switching having occurred:
- Reconstructor trained to generate realistic supervisor input by sampling from the latent space
- GAN trained as a 2-player game loss function, between detector and reconstructor as:

$$\Theta_d^*, \Theta_g^* = \arg \min_g \max_d J(\Theta_d, \Theta_g)$$

- defense-GAN works upstream of the supervisor in the cyber system layer:



Example: *N*-robots in a network

- Consider a network of N nonholonomic robots following Dubin's equations:

$$\dot{x}_i = v_i \cos \theta_i, \quad \dot{y}_i = v_i \sin \theta_i, \quad \dot{\theta}_i = \omega_i$$

- Linearization for i^{th} robot wrt state (x_i, y_i, θ_i) and control input (v_i, ω_i)
- Supervisory controller intends to flock the robots; switches among a bank of gains:

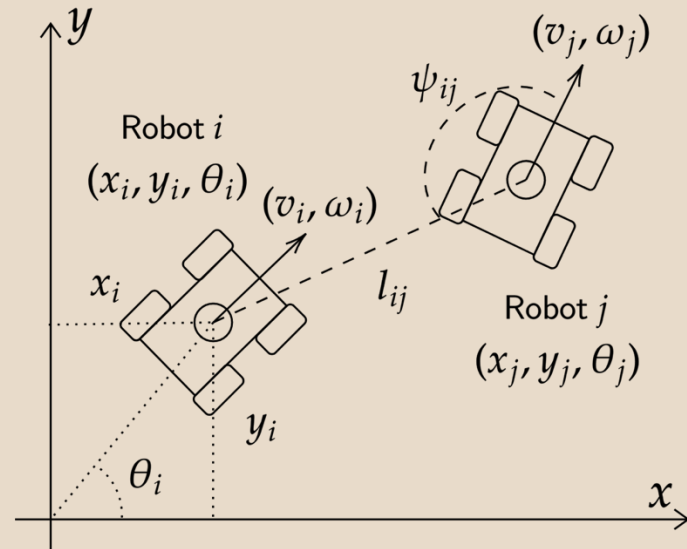
$$\left\{ K_{(1)}^i, K_{(2)}^{i,j}, K_{(3)}^i, K_{(4)}^{i,j} \right\}_{i,j=1}^N$$

- Supervisor consists of 4 discrete states [15]:

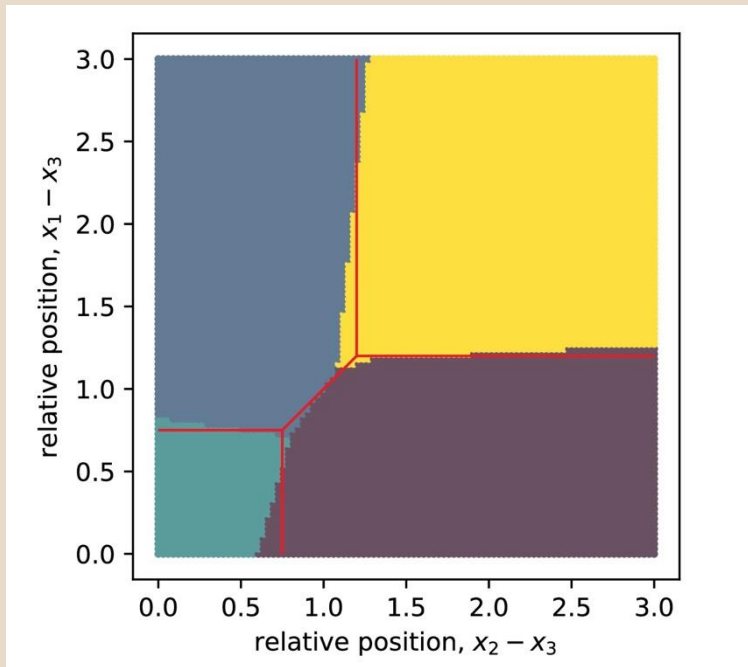
Separation Bearing Control ($SB_{ij}C$, robot i follows j),
 Separation Separation Control ($S_{ij}S_{jk}C$, robot k follows i, j),
 Separation Distance-to-Obstacle Control (SD_oC), and
 Autonomous navigation ($u_i \equiv 0$)

- Each robot queries for its active controller by communicating relative distance l_{ij} with the supervisor
- Supervisor returns the relevant controller to be used by the given robot:

$$SB_{ij}C, S_{ij}S_{jk}C, SD_oC, \text{ or } u_i \equiv 0$$



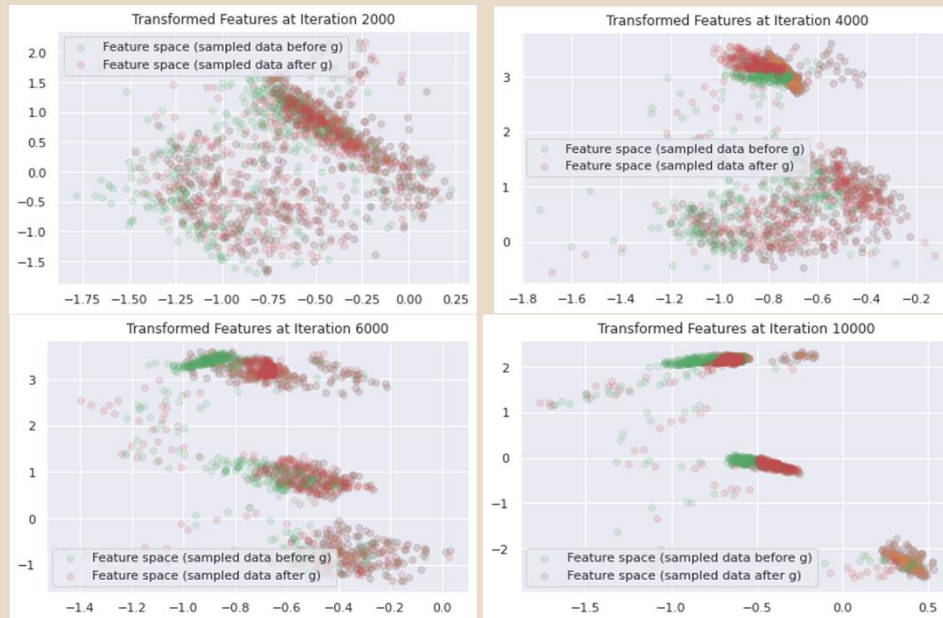
The attacker intends to inject a_k to robot 3's state. To do so, the attacker makes open-loop queries with the supervisor. The attacker makes 500 queries to infer discrete state transition map $\hat{\delta}$



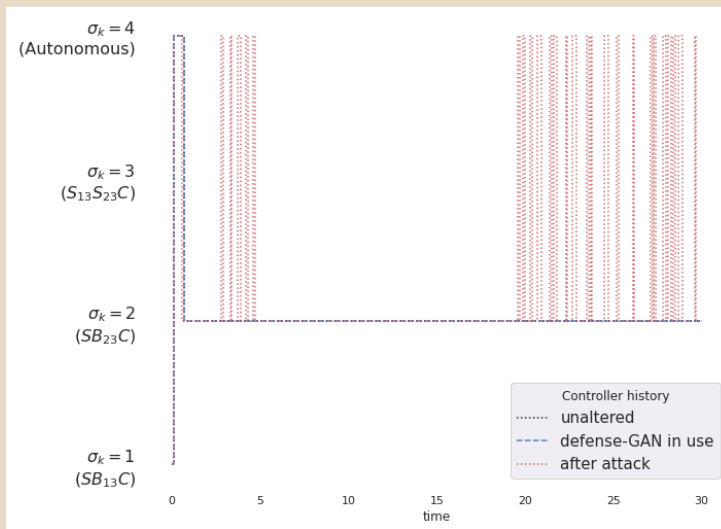
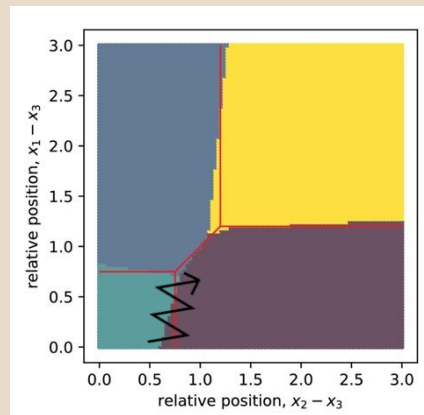
true guard conditions that partition state space $\mathcal{X}$ for the 4 operational controllers, v/s partition obtained by transition map $\hat{\delta}$ obtained with a neural network;

The detector and reconstructor are both neural networks, trained against each other over 10,000 epochs

- The detector has a single output unit to estimate attack probability $P(\sigma_k \neq \tilde{\sigma}_k; z|\tilde{x}_k)$
- Reconstructor generates input samples
- Detector tries to classify samples into $\sigma_k \neq \tilde{\sigma}_k$ and $\sigma_k = \tilde{\sigma}_k$

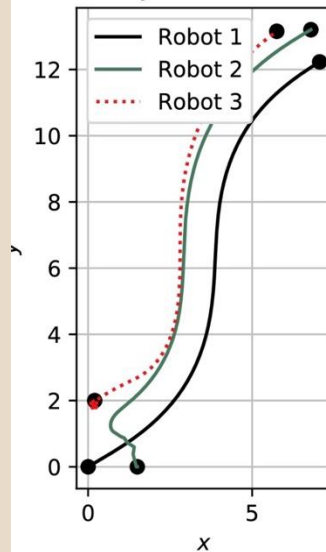


Intermediate results produced by the reconstructor, visualized in the feature space;
tighter clustering in feature space implies reconstructor getting better as it learns against the detector

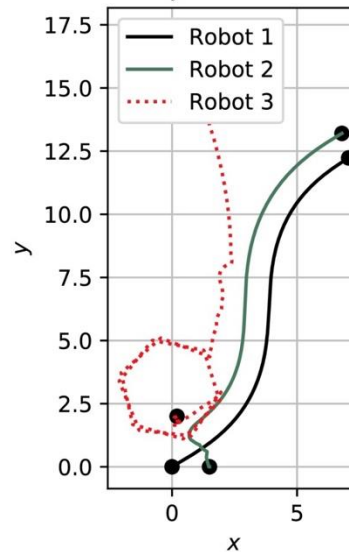


discrete state for
unattacked trajectory
of robot-3 v/s attacked
trajectories and using
defense-GAN

Robot trajectories over time



Robot trajectories over time



(left) unattacked trajectories
for the three robots

(right) robot-3's discrete
state attacked as shown in
the figure below

Defense-GAN able to detect 97.6% of attack occurrences over a run of 10,000 trajectories

Mixed Monotone Reachability for Data-Driven System ID

Motivation

- HJ-type reachability cannot be performed real-time
- Polytopic reachable set computation can be done for a known model
- Zonotope, or interval reachability is faster to compute, but with a known model
- *Mixed monotone reachability generalized interval reachability in a computationally inexpensive way*

How can we compute reachable sets for unknown models from data?

Problem Formulation

- Consider an unknown, nonlinear dynamical system with state $x \in \mathcal{X}$ and admissible control $u \in \mathcal{U}$
- If the state transition map for the unknown dynamics is given by $\Phi^f(\tau; x, u)$ that takes state x under control u and dynamics f , in time τ
- The reachability problem associated with the dynamics above is to find

$$\begin{aligned}\mathcal{R}^f(T; x_0) &:= \{x(T) \in \mathcal{X} : \dot{x} = f(x, u), x_0 \in \mathcal{X}_0, u \in \mathcal{U}\} \\ &= \left\{ \Phi^f(T; x_0, u) \in \mathcal{X} : x_0 \in \mathcal{X}_0, u \in \mathcal{U} \right\}\end{aligned}$$

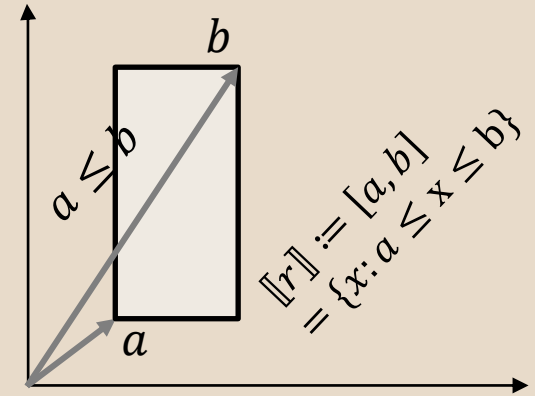
- Assume that sampled trajectory data from the dynamics is available in the form $\{x_i, \dots x_{i+N}\}$ and $\{u_i, \dots u_{i+N}\}$

For some approximate the dynamics $\tilde{f}$, given the data, can we compute $\mathcal{R}^{\tilde{f}}$ s.t.

$$\mathcal{R}^f \subseteq \mathcal{R}^{\tilde{f}}?$$

Consider a simpler problem where $\tilde{f}$ is a polynomial

- Define partial order for vectors in $\mathbb{R}^n$ where $a \leq b$ if $a_i \leq b_i$ component-wise.
- Also define an extended hyperrectangle $[a, b] = \{x: a \leq x \leq b\}$ for two vectors $a \leq b$. A bounded extended hyperrectangle is a hyperrectangle $r := (a, b) \in \mathbb{R}^{2n}$, $\llbracket r \rrbracket := [a, b]$
- For some $F(u, v)$, mapping F is mixed-monotone if the Jacobian increases in u , and decreases in v



$$F_u(u, v) \in [\underline{J}_u, \overline{J}_u], F_v(u, v) \in [\underline{J}_v, \overline{J}_v], \forall u \in \mathcal{U}, v \in \mathcal{V}$$

$$(\underline{J}_u)_{i,j} > -\infty \text{ or } (\overline{J}_u)_{i,j} < \infty, \forall i \neq j, \text{ and}$$

$$(\underline{J}_v)_{i,i} > -\infty \text{ or } (\overline{J}_v)_{i,i} < \infty, \forall i, j$$

- Dynamics $f(u, v)$ is mixed-monotone w.r.t. some decomposition function $d(u, v, u, v)$ in a higher dimension if:

$$F(u, v) = d(u, v, u, v),$$

$$(d_{u_j}(u, v, \hat{u}, \hat{v}))_i \geq 0, (d_{\hat{u}_j}(u, v, \hat{u}, \hat{v}))_i \leq 0, \forall i, j$$

$$(d_{v_j}(u, v, \hat{u}, \hat{v}))_i \geq 0, (d_{\hat{v}_j}(u, v, \hat{u}, \hat{v}))_i \leq 0, \forall i, j$$

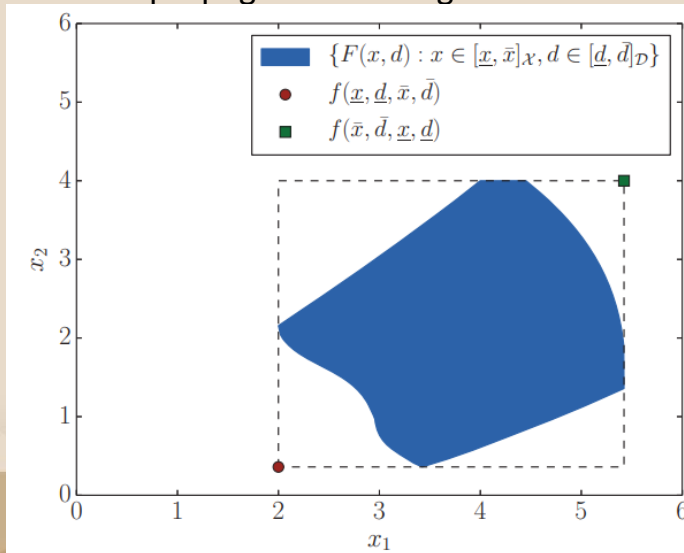
- From [7,8], such a d always exist if $f(x, u)$ is a polynomial $d(u, v, u, v)$

Interval Reachability for Polynomials

- Let $f(x, u)$ be some polynomial dynamics and the sets $\mathcal{X}_0$ and $\mathcal{U}$ be given by hyperrectangles $[\underline{x}_0, \bar{x}_0]$ and $[\underline{u}_0, \bar{u}_0]$
- There exists some decomposition $d(x, u, x, u)$ s.t. f is mixed-monotone w.r.t. d
- Define an artificial system with twice as many states as $\begin{bmatrix} x \\ \hat{x} \end{bmatrix}$ obtained by fixing the control u at its hyperrectangular bounds s.t.

$$\begin{bmatrix} \dot{x} \\ \dot{\hat{x}} \end{bmatrix} = \Theta(x, \hat{x}) := \begin{bmatrix} d(x, \underline{u}_0, \hat{x}, \bar{u}_0) \\ d(\hat{x}, \underline{u}_0, x, \bar{u}_0) \end{bmatrix}$$

- Then the following holds for the deterministic dynamics Θ with a higher dimension: $\mathcal{R}^f(T; x_0) \subseteq \llbracket \Phi^\Theta(T; (\underline{x}_0, \bar{x}_0)) \rrbracket$ [7]**
- f is polynomial $\Rightarrow$ interval over-approximations of the reachable set can be found by a deterministic state propagation of a higher dimensional system Θ

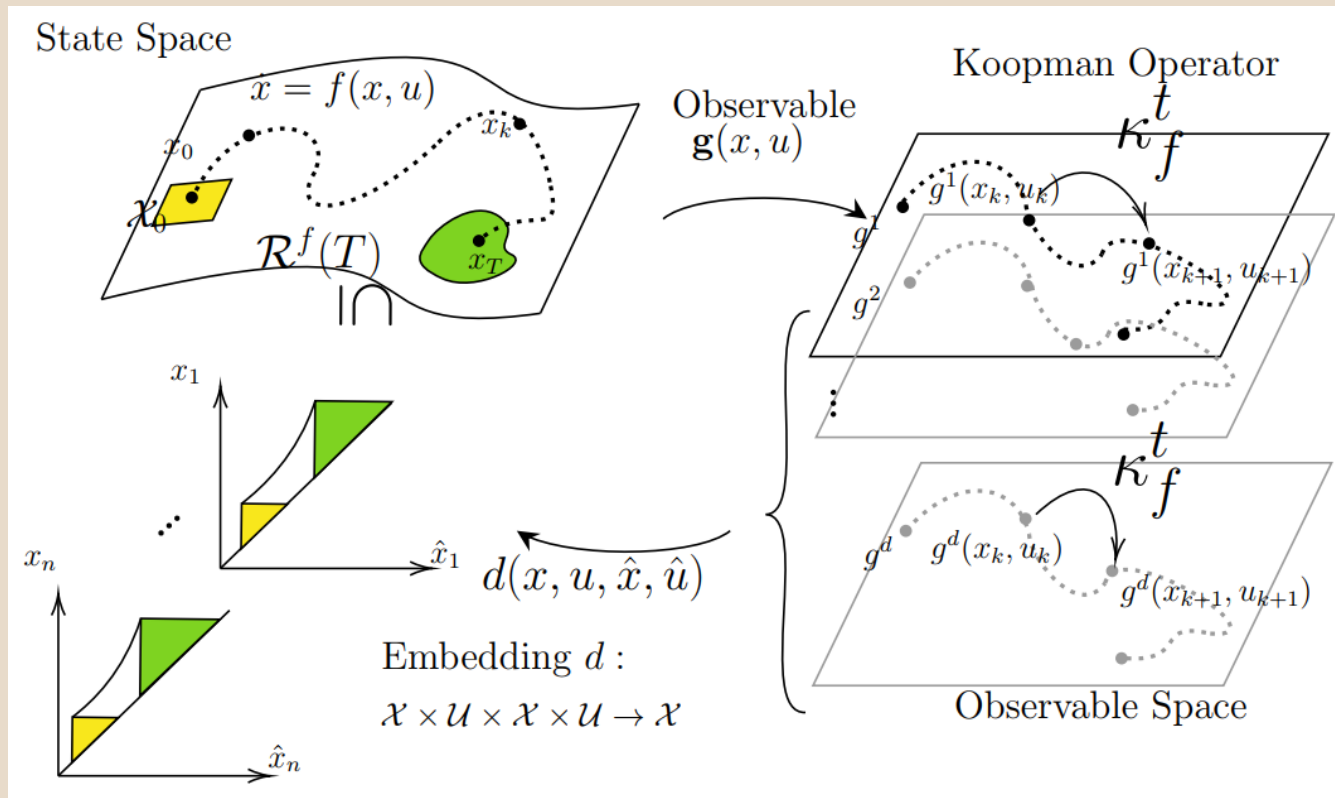


We can bound $F(x, d)$ when x and d are confined to lie within a given rectangle, by evaluating the decomposition function at two points. [8]

The decomposition here is evaluated at the rectangle corners as: $f(\underline{x}, \underline{d}, \bar{x}, \bar{d})$ and $f(\bar{x}, \bar{d}, \underline{x}, \underline{d})$

Extended Dynamic Mode Decomposition

- Given the sample trajectory data, we can find polynomial approximation to unknown dynamics f using Koopman Operator theory [9]
- Koopman operator takes dynamics in state space to some observable space $g(x, u)$,
 - the observable can be thought to be some 'feature' space
 - Consider g to be polynomial observables



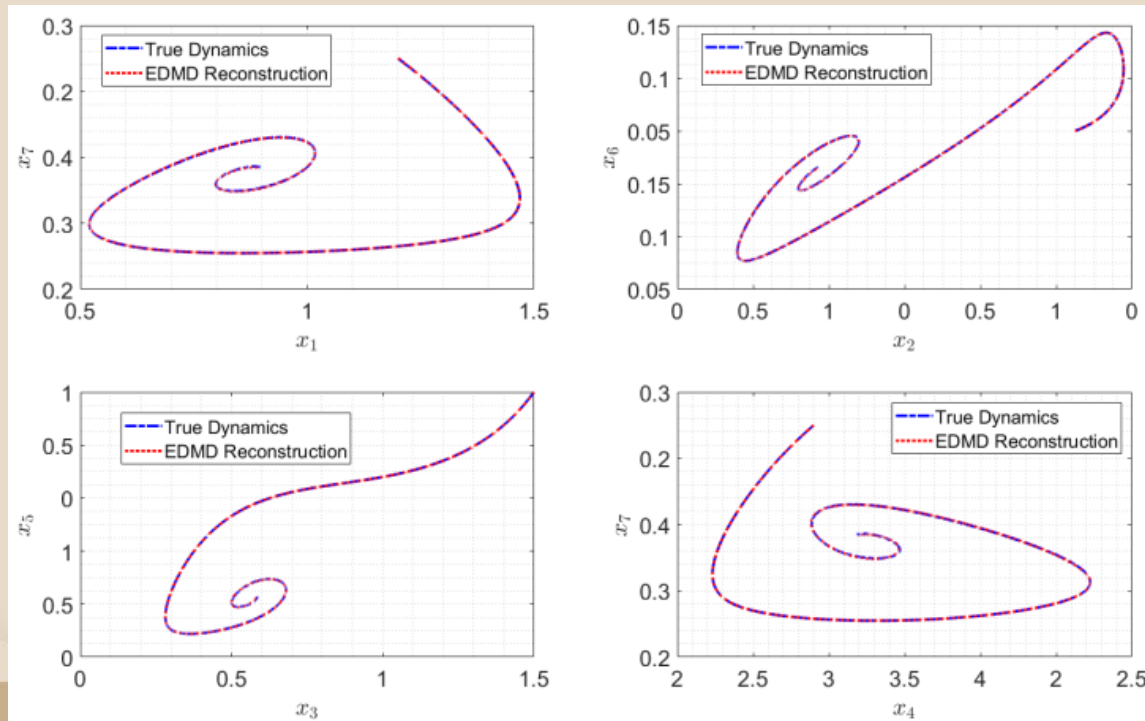
Data $\rightarrow$ polynomial model using EDMD $\rightarrow$ decomposition $\rightarrow$ interval over-approximations of reachable set

Approximating Reachable Set for high-D unknown dynamical model

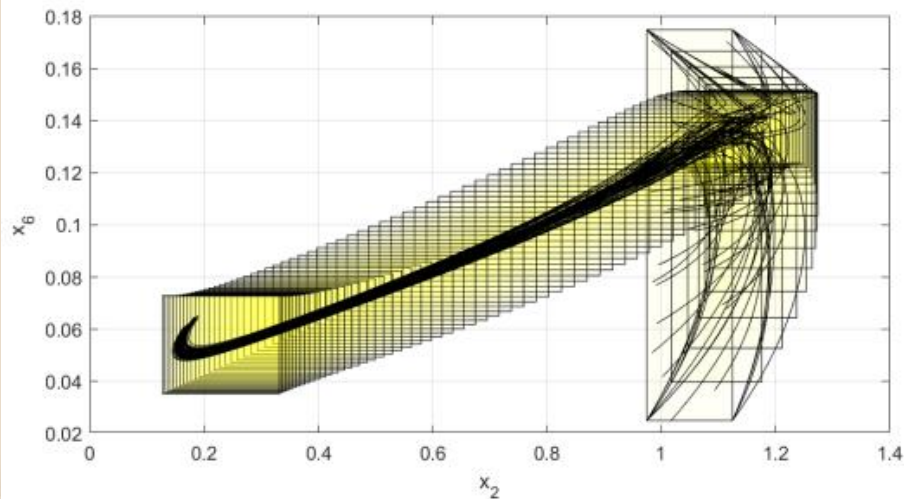
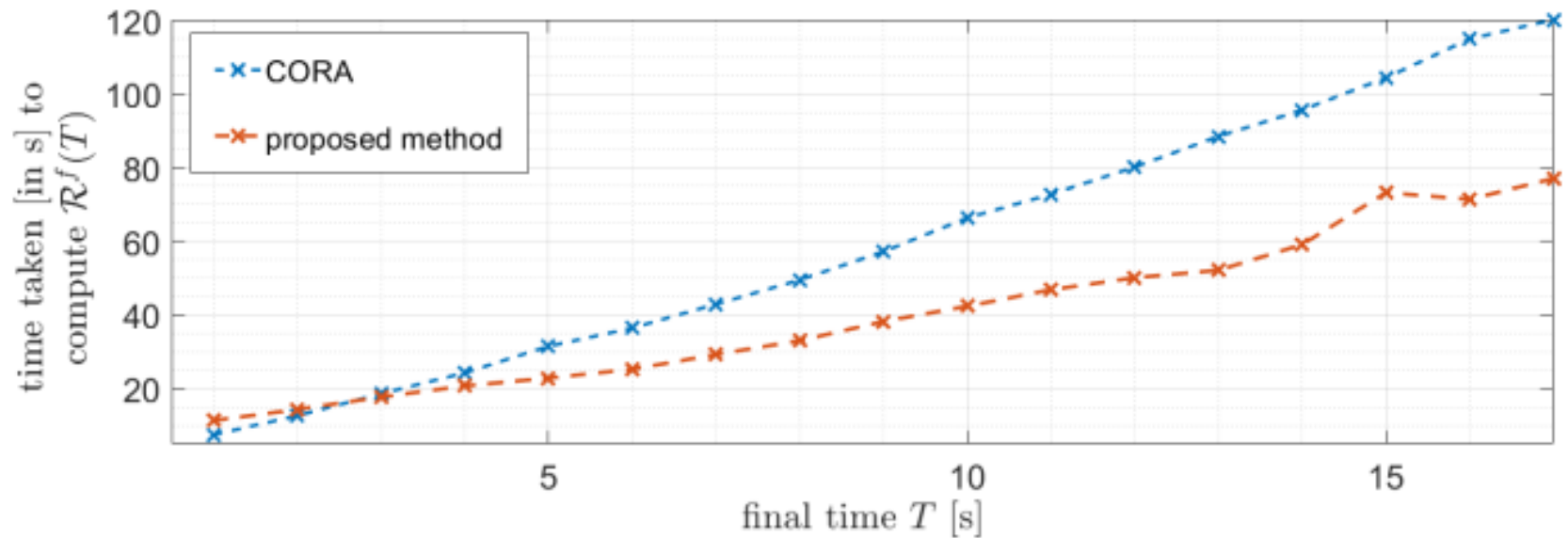
Laub-Loomis model – 7D model used for enzymatic activities as

$$\begin{aligned}\dot{x}_1 &= 1.4x_3 - 0.9x_1, & \dot{x}_2 &= 2.5x_5 - 1.5x_2, \\ \dot{x}_3 &= 0.6x_7 - 0.8x_2x_3, & \dot{x}_4 &= 2 - 1.3x_3x_4, \\ \dot{x}_5 &= 0.7x_1 - x_4x_5, & \dot{x}_6 &= 0.3x_1 - 3.1x_6, \\ \dot{x}_7 &= 1.8x_6 - 1.5x_2x_7\end{aligned}$$

We first estimate polynomial approximations using trajectory data by EDMD (using polynomial observable functions) as:



Approximate Reachable Sets



- Proposed method is substantially faster than existing SOTA method
- Proposed method trades computational complexity for over-approximation of $\mathcal{R}^f$
- Fully data-driven
- ***Approximations are not tight***

Polytopic Reachability: Optimal Control Problem

Suppose $\dot{x} = Ax + u$ where $u \in \Omega$ and $x(0) \in \mathcal{X}_0$. Note the following:

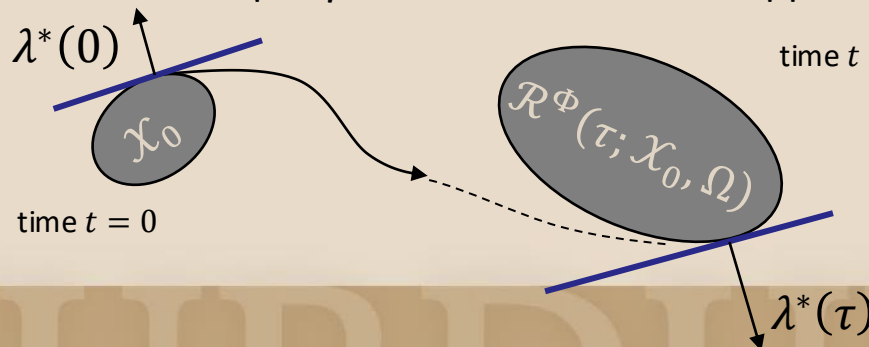
- If Ω and $\mathcal{X}_0$ are convex, compact $\Rightarrow \mathcal{R}^\Phi(\tau; \mathcal{X}_0, \Omega)$ is also convex, compact
- Reach set at time τ can be written as

$$\mathcal{R}^\Phi(\tau) = e^{\tau A} \mathcal{X}_0 + \int_0^\tau e^{(\tau-s)A} \Omega ds = e^{\tau A} \mathcal{X}_0 + \lim_{\Delta t \rightarrow 0} \sum_0^{\lfloor \tau/\Delta t \rfloor} \Delta t e^{i\Delta t A} \Omega$$

- In the limit, $\mathcal{R}^\Phi$ is smooth, convex and compact
- Symbolic representation of $\mathcal{R}^\Phi$ is hard to obtain, but since it is compact, we can obtain a polytopic approximation
- Since $\mathcal{R}^\Phi$ is convex in $\mathbb{R}^n$, to obtain a polytopic approximation within δ of $\mathcal{R}^\Phi$ as: $\mathcal{R}^\Phi \subset \text{convex hull}\{x^1, \dots, x^{n_P}\} + \mathbb{B}(\delta)$ and $\cap_i \{\langle c_i, x \rangle \leq \gamma_i\} \subset \mathcal{R}^\Phi + \mathbb{B}(\delta)$, we need $n_P = \mathcal{O}(\delta^{-(1-n)})$ number of points

Maximum Principle

- If (c_0, γ_0^*) supports $\mathcal{X}_0$, and a control input is applied from a convex set $u(\tau) \in \Omega$, by Maximum Principle, $\gamma^*(\tau) = \langle \lambda^*(\tau), x^*(\tau) \rangle$ supports $\mathcal{R}^\Phi(\tau; \mathcal{X}_0, \Omega)$



- What control input pushes $x^*(0)$ furthest away from initial plane?
- $\lambda^*(\tau)$ denotes the normal vector of the hyperplane at time $t = \tau$

To confirm that $\lambda^*(s)$ supports $\mathcal{R}^\Phi(s)$, observe:

- $$\begin{aligned} \frac{d}{ds} \langle \lambda^*(s), x^*(s) \rangle &= \langle \dot{\lambda}^*(s), x(s) \rangle + \langle \lambda^*(s), \dot{x}^*(s) \rangle \\ &= -\langle \lambda^*(s), Ax(s) \rangle + \langle \lambda^*(s), Ax(s) + Bu(s) \rangle, \\ &= \langle \lambda^*(s), Bu(s) \rangle \leq \langle \lambda^*(s), Bu^*(s) \rangle = -\langle \lambda^*(s), Ax^*(s) \rangle + \langle \lambda^*(s), Ax^*(s) + Bu^*(s) \rangle, \\ &= \frac{d}{ds} \langle \lambda^*(s), x^*(s) \rangle = \frac{d}{ds} \gamma^*(s), \text{ which implies } \langle \lambda^*(s), x(s) \rangle \leq \langle \lambda^*(s), x^*(s) \rangle = \gamma^*(s) \end{aligned}$$
- The, the Symbolic representation of $\mathcal{R}^\Phi$ is approximately given by $\lambda_i^*(\tau), c_i^*(\tau), \gamma_i^*(\tau)$ at some time $t = \tau$, for hyperplanes $i = 1, \dots, n_p$
- Where the i^{th} hyperplane's evolution can be given by the optimal control problem:
 $\dot{x}_i^*(\tau) = Ax_i^*(\tau) + Bu_i^*(\tau), x_i^*(0) = \text{initial points of contact of the hyperplane}$
 $\dot{\lambda}_i^* = -A^T \lambda_i^*(\tau), \lambda_i^*(0) = c_i$, and the control input to push the point furthest is simply given as,
 $u_i^*(\tau) \in \operatorname{argmax}\{\langle \lambda_i^*(s), Ax_i^*(\tau) + Bu \rangle \mid u \in \Omega\}$
- $\lambda_i^*(\tau)$ has a closed form solution $\exp\{-\tau A^T\} c_i$, and For a time-varying system, $\lambda_i^*(\tau) = \Phi^\lambda(\tau; 0) c_i$

Tightness of Approximation

- The approximation thus found provides both, upper and lower bounding polytopes, i.e.,

$$\operatorname{convex hull}\{x_1^*(\tau), \dots, x_{n_p}^*(\tau)\} \subset \mathcal{R}^\Phi(\tau; \mathcal{X}_0, \Omega) \subset \bigcap_i \{\langle \lambda_i^*(\tau), x(\tau) \rangle \leq \gamma_i^*(\tau)\}$$

- There does not exist another polytope with n_p hyperplanes that can reside within the upper and lower polytopes above $\Rightarrow$ the approximation is tight
- This generalizes nicely for a computationally inexpensive method to approximate $\mathcal{R}^\Phi$ for linear(-izable) systems

Extending Distributed Reachable Set Computation: $\mathbb{A} \rightarrow \mathbb{A}(t)$

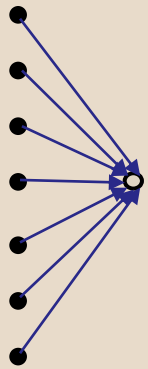
Let $\mathcal{G}$ be the time-varying, undirected graph, given by $\mathcal{G}(t) = \{\mathcal{E}(t), \mathcal{V}(t)\}$

- $\rightarrow$ the graph adjacency varies as $\mathcal{A}(t) \Rightarrow$ we get a linear time-varying system $\mathbb{A}(t), \mathbb{B}$
- Since $\mathcal{G}(t)$ is undirected, $\mathcal{A} = \mathcal{A}^T$ at all times $\Rightarrow \mathbb{A}(t) = \mathbb{A}(t)^T$
- To propagate the costate and state, we require to calculate $\Phi^{\mathbb{A}}(\tau; 0)$
- Let each agent be aware of a (possibly empty) partition of the basis vectors e_i such that $\{e_i\}$ spans $\mathcal{X}$
- To compute $\Phi(\tau; t_0)$, note the following:
 - Let $\hat{e}(\tau)$ solve the differential equation $\dot{x} = \mathbb{A}(t)x$ for initial condition $x(t_0) = e_i$
 - Accumulating $\hat{e}(\tau)$ provides columns of the matrix $\Phi(\tau; t_0) = [\hat{e}_1(t), \dots, \hat{e}_i(t), \dots, \hat{e}_{n_x \cdot n_p}(t)]$
 - For some time t , the state-transition map can be found using:

$$\Phi(t; \tau) = \Phi(t; t_0)\Phi(\tau; t_0)^{-1}$$
- The above can be carried out in a distributed manner for $\mathcal{G}(t)$ strongly connected at all times

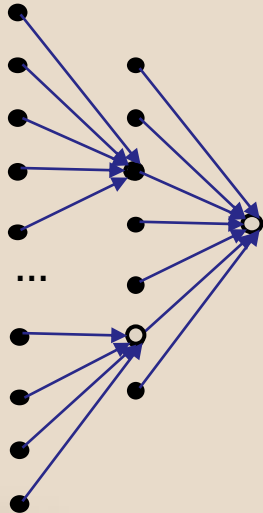
Basic NNs

Neural networks are bio-inspired computational tools — for our purposes, a NN is an arbitrary function approximator. A caricature of a simple NN architecture can be coupled neurons as



Here the state of an individual neuron is either at rest, or firing, as $\text{input} > \theta: S \rightarrow 1$ and $\text{input} < \theta: S \rightarrow 0$

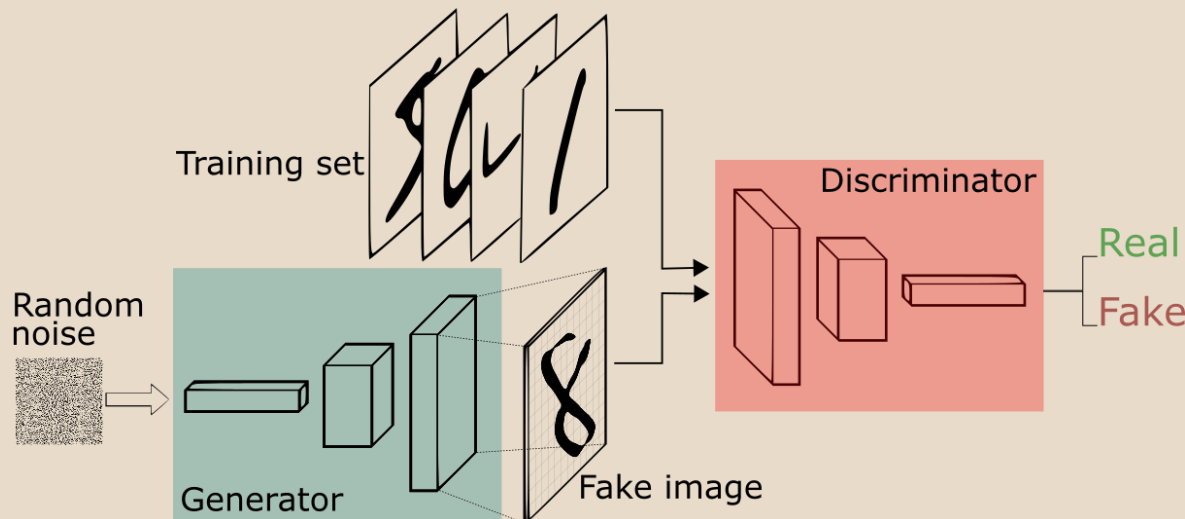
- Let the individual signals add linearly as $w_1 S_1 + \dots + w_N S_N$ for the N neurons
- The effect of input of a neuron l firing is $S_l \rightarrow 1$: $\text{input} \rightarrow \text{input} + w_l$, where $w_l > 0 \leftrightarrow \text{input} \uparrow$ (excitation), and $w_l < 0 \leftrightarrow \text{input} \downarrow$ (inhibition)
- The l^{th} neuron updates as: $S_l(t+1) = 1$ if $\sum_i w_{li} S_i(t) > \theta_l$ (and $S_l(t+1) = 0$ if $\sum_i w_{li} S_i(t) < \theta_l$)
- In practice, there would be multiple layers of such neurons, where the output of the entire network is to learn a function approximation (say, MSE given some data points)



- Let the output of the entire network be some $y(x_{in}; W, \Theta)$ for accumulated parameters Θ and weights W
- The network can be trained as $\Theta^*, W^* = \text{argmin} \left\{ \sum_i \|y_{data,i} - y(x_{data,i}; W, \Theta)\|^2 \right\}$
- The actual training is done via stochastic gradient descent (SGD)
- The network is very sensitive to W, Θ initializations, and therefore, SGD is performed over multiple batches of data (each SGD run comprising an “epoch” of training the NN)
- The network depth (number of layers) and height (number of neurons) are design parameters; sometimes an activation function can also be employed (some bounded $\sigma(S(t))$ to the linearly combined states of individual neurons)
- **NNs of such architecture are: a) universal information processing systems** (can perform any computation that can be performed by a conventional digital computer), **and b) universal function approximators** (for specific activation functions σ)

Basic GANs

- *Generative Adversarial Networks (GANs) are NNs with a special architecture type.*
- *A single GAN is composed of two individual NNs, each trained separately against each other, in a 2-player game → min-max loss function-based training*
- *The generator is a special NN that attempts to memorize given data patterns → tries to generate an x_G by learning $\mathbb{P}(x; W_G, \Theta_G)$, given data $x_{data} \sim \mathbb{P}(x)$ (or an autoencoder type architecture)*
- *The discriminator is a special NN that is a binary classifier → given input x_G , is x_G really sampled from some $\mathbb{P}(x)$ or not? (a simple binary classification NN)*
- *→ NN_G and NN_D are trained against each other as some: $\Theta_d^*, \Theta_g^* = \arg \min_g \max_d J(\Theta_d, \Theta_g)$*
- *For example, the famous MNIST (numerical classification dataset) might have a GAN that looks like:*



- *The generator would learn by sampling $x_{sample} \sim \mathcal{U}$ over random noise (sample image size)*
- *The discriminator will learn by using both data, and the input from generator*
- *Training GANs is, therefore, much slower than a single discriminator (or autoencoder)*
- *GAN training is performed over multiple epochs, until the 2-player game has “converged enough”*